

基于变物性与收缩效应的圆柱药材热湿迁移模型

摘要

针对圆柱形药材烘干中环境边界时变、内部温湿分布不均、物性非线性及几何收缩相互影响的问题，本文构建统一的变物性径向热湿有限体积框架。以 PCHIP 重构附件中的环境量和半径曲线，以 Fourier–Fick 方程描述内部传递，采用节点中心有限体积法与全隐式格式保证离散守恒和长时计算稳定，并通过 Picard 迭代处理状态相关物性；进一步以材料坐标下的固定域映射描述收缩边界，以全截面首达时间统一衡量干燥终点。

针对问题一：在固定半径和给定物性下，建立 **Fourier–Fick 基准模型**，将时变环境量作为双 **Robin 边界输入**并逐秒求解。利用中心圆盘控制体处理轴线边界，结合表面加密网格解析外层梯度。1800 s 时中心与表面温度分别为 **33.577 °C 和 36.786 °C**，水分浓度分别为 **2.550 和 1.510 kg/kg**。结果表明预热初期水分主要从表层逸出，内部水分浓度仍接近初值。

针对问题二：在基准模型上引入 $\rho(C)$, $c_p(C)$, $k(C)$ 与 $D(T, C)$ ，构造**温度–水分状态耦合的非线性扩散模型**，每个时间层交替更新温度、扩散系数和水分场，以双重收敛判据保证物性与场量自洽，刻画升温促进扩散与失水抑制扩散的竞争。3 h 时中心与表面温度达到 **49.849 °C 和 49.967 °C**，水分浓度降至 **1.766 和 1.008 kg/kg**。此时径向温差已很小，而水分梯度仍然明显，后续过程主要受内部传质控制。

针对问题三：将问题二模型延伸至长期干燥，以 $\max_r C(r, t) \leq 0.15 \text{ kg/kg}$ 定义全截面达标，并对跨越阈值的相邻时间层实施线性事件插值。通过径向单调性与含端面传递的二维计算核验控制位置，区分输出间隔与事件定位精度。根据末段观测的稳定水平，假设 4 h 后环境保持为 $T_e = 50 \text{ °C}$ 、 $C_e = 0.05 \text{ kg/kg}$ 。**全域首达检测得到干燥时间为 57.472 h**，达标瞬间中心和表面水分浓度分别为 **0.150 和 0.053 kg/kg**。全程中心均为最大值，说明仅依据表面含水率停机将低估内部干燥时间。

针对问题四：建立**收缩移动边界热湿耦合模型**，通过 $\xi = r/R(t)$ 将时变物理区域映射到固定计算域；同时将半径收缩引起的扩散距离缩短及表面交换变化纳入模型。在均匀材料收缩与干骨架守恒假设下，使材料速度与网格速度一致，并以同物性对照分离几何贡献。**移动边界模型得到干燥时间为 51.088 h**，达标半径为 **1.200 cm**；同物性固定半径方案需 129.845 h，收缩使时长缩短 60.7%。进一步采用**移动网格验证**，两种坐标实现的结果在浮点舍入量级内一致；**COMSOL 有限元复核**经表层网格加密后，最大水分差降至 $4.54 \times 10^{-5} \text{ kg/kg}$ ，支持既定模型的数值可靠性。

数值收敛、长期二维核验及移动网格交叉验证表明模型结果具有良好的稳定性与可靠性。灵敏度分析显示，环境温度是影响干燥时长的主要外部因素，所建模型可用于描述变物性与收缩条件下圆柱药材的热湿迁移过程。

关键字： 药材烘干 Fourier–Fick 模型 有限体积法 热湿耦合 移动边界

一、问题重述

1.1 问题背景

药材烘干过程同时包含内部传热和水分迁移。外部温度变化首先作用于药材表面，热量向内部传递；水分则在浓度梯度和表面传质作用下由内部向外迁移。由于热扩散和水分扩散的时间尺度不同，温度场和水分场通常具有明显的径向非均匀性，表面状态不能代表整个截面的状态。因此，需要建立描述药材内部温度和水分浓度时空分布的非稳态模型，并以全截面状态作为干燥完成的判断依据。

本题研究一根长 25 cm、初始半径 2 cm 的近似圆柱形药材，初始温度和水分浓度分别为 28 °C 和 2.55 kg/kg。已知条件依次引入时变环境、状态相关物性、全截面含水率约束以及半径收缩。因而四个问题分别对应恒定热物性、变物性耦合、全域达标判定和移动边界四个层次；其中问题一仍保留水分扩散系数 $D(C)$ 的非线性，模型在统一热湿传递框架下逐步增加相应的物理因素。

1.2 问题提出

根据题目给定的几何尺寸、初始状态、环境条件及物性关系，本文需要获得药材内部温度和水分浓度的时空分布，并确定全截面达到含水率要求的时间。四问的区别在于逐步增加物性变化、全域约束和几何收缩等因素，因此采用由简到繁的统一建模路线。

问题一：首先需要刻画预热阶段药材内部热量与水分的径向迁移规律。由于药材内部存在明显的温度和水分梯度，应建立圆柱坐标下的非稳态传热传质模型，并结合时变环境边界求得温湿场，为后续各问提供统一的基础模型和数值求解框架。

问题二：在基础模型上进一步考虑物性随温度和水分状态变化的影响，使导热、储热和水分扩散过程相互耦合。由此需将问题一的基准模型扩展为变物性的非线性热湿迁移模型，以分析整个烘干过程中温度场与水分场的动态演化及其控制机制。

问题三：在获得长期水分迁移规律后，需要进一步解决“何时完成烘干”的判定问题。将“药材各处水分均低于规定阈值”转化为空间最大水分浓度的首达条件，并据此确定控制位置及最终烘干时间，从而避免仅依据表面或平均水分作出过早停机判断。

问题四：实际烘干过程中药材半径持续收缩，使计算区域由固定域转化为移动域。为此需在变物性热湿模型中进一步引入时变几何边界，通过固定域映射处理收缩过程，并分析扩散距离缩短对烘干效率的影响；同时采用独立移动网格计算以及 COMSOL 物理仿真对移动边界结果进行交叉验证。

二、问题分析

四问共享“附件数据重构—内部热湿传递—表面对流交换—数值求解—结果验证”的主链条，但各问新增困难不同。问题一解决基础的时变边界扩散；问题二处理系数随解变化引起的非线性；问题三引入全域阈值首达事件；问题四再处理随时间收缩的空间区域。按照这一依赖关系建立同一套守恒离散程序，可以避免各问采用不同数值口径造成的结果不可比。总体技术路线如图 1 所示。

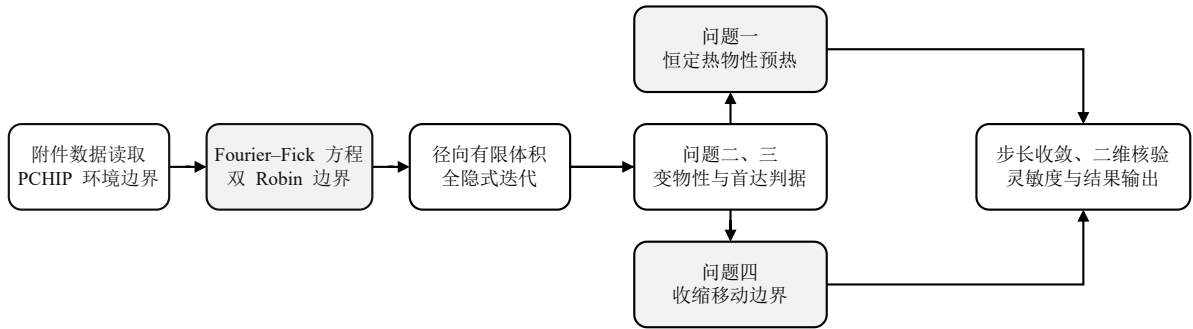


图 1 四问递进建模与验证技术路线

2.1 问题一的分析

问题一的半径和主要热物性给定为常数，但热量和水分仍需在药材内部传递。热、质扩散特征时间不同，且表面对流边界不能忽略，因此不能用截面平均量描述预热过程。本文建立一维径向热湿传递模型，保留水分扩散系数 $D(C)$ 的非线性；由附件重构时变环境，在中心和表面分别设置对称与 Robin 边界，并采用有限体积法求解预热阶段的温湿分布，为后续各问提供统一的离散框架。

2.2 问题二的分析

问题一的恒定热物性假设不能反映干燥过程中密度、比热和导热系数随状态变化的影响，水分扩散系数则已保留 $D(C)$ 的非线性。问题二进一步引入状态相关热物性及 $D(T, C)$ ，形成温度场与水分场相互影响的非线性耦合模型。由于系数依赖待求状态，采用分步 Picard 迭代更新温度、水分与物性，获得同一时间层的自洽解，再比较中心与表面的响应。

2.3 问题三的分析

问题三要求所有位置的水分浓度均不超过 0.15 kg/kg ，因此表面浓度或截面平均浓度都不能作为停机依据。本文在问题二模型上延长计算时间，以全域最大水分浓度的首

次达标作为事件判据；通过全过程剖面检查确定控制位置，并在阈值附近进行时间插值，以提高达标时间的定位精度。

2.4 问题四的分析

问题四中药材半径随时间变化，固定空间区域和固定网格不能直接描述材料边界的移动。本文假设药材均匀径向收缩，利用 $\xi = r/R(t)$ 将移动域映射至固定区间，并在问题二的变物性热湿模型上更新几何尺度。为区分收缩本身的影响，另设使用相同物性和环境的固定半径对照模型；再用物理坐标移动网格与 COMSOL 有限元计算进行交叉验证。

三、模型假设

根据题给条件，作如下假设：

1. **介质与几何**。药材为均匀、各向同性的连续介质，初始温湿场均匀且轴对称；主模型仅考虑中截面径向传递，端面影响另用二维模型检验。
2. **传递机制**。热量与水分分别由导热和 Fick 扩散传递，耦合仅通过题给状态相关物性体现；忽略内热源、辐射、内部宏观对流、重力渗流及托盘接触传热。
3. **表面交换与浓度口径**。表面采用对流换热、传质边界，四间均取 $h_T = 25 \text{ W}/(\text{m}^2\text{K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ ；药材与环境水分浓度均按同一干基比值处理。
4. **蒸发潜热**。沿用题给热传递模型，不单独计入蒸发潜热项，其对温度及干燥时间的影响作为模型局限。
5. **环境延拓**。根据附件一末段观测的稳定水平，假设超过其覆盖的 14400 s 后，环境温度 and 水分浓度分别保持为 $50.000 \text{ }^\circ\text{C}$ 、 0.05000 kg/kg 。
6. **收缩闭合**。问题四长度不变，材料点沿径向同比例收缩；干骨架质量守恒且密度在截面内均匀，满足 $\rho_d R^2 = \rho_{d0} R_0^2$ 。题给 ρc_p 仅作有效储热系数。

四、符号说明

本文主要符号及其单位见表 1。温度函数中的绝对温度统一写为 $T_K = T + 273.15$ ，其余温差仍可直接用摄氏度计算。

表 1 主要符号及含义

符号	含义	单位	备注
r, z	径向、轴向物理坐标	m	主模型只保留 r
$R(t), L$	瞬时半径、圆柱长度	m	$L = 0.25$
T, T_e	药材温度、环境温度	°C	扩散系数中换为 K
C, C_e	药材、环境水分浓度	kg/kg	干基比值
ρ, c_p	密度、比热容	kg/m ³ , J/(kg K)	状态相关
k, D	导热系数、水分扩散系数	W/(m K), m ² /s	状态相关
h_T, h_m	表面传热、传质系数	W/(m ² K), m/s	Robin 边界
ξ	固定域材料坐标	1	$\xi = r/R(t)$
V_i, G_f	控制体体积、界面导通系数	m ² , 依场量而定	按单位轴向长度
$\Delta r_i, \Delta t$	局部空间、时间步长	m, s	表面加密网格, 1 s
n, m	时间层、Picard 迭代编号	1	非负整数
t_d	全截面达标首达时刻	s 或 h	$\max C \leq 0.15$

五、模型准备

5.1 几何降维与初始条件

药材的初始几何及坐标约定见图 2。取一端面中心为原点 O ， z 轴沿药材轴线， x 、 y 轴位于端面内；圆柱长 $L = 25$ cm，初始半径 $R_0 = 2$ cm。中截面中心记为 O_m ，径向距离 r 从该点起算，周向角 θ 从正 x 轴起算。图中热量传入箭头示意预热阶段，水分箭头表示向外迁移。

在圆柱坐标 (r, θ, z) 中，轴对称条件使 $\partial/\partial\theta = 0$ 。主模型进一步取中截面 $z = L/2$ ，忽略该处的轴向梯度，计算域为 $0 \leq r \leq R(t)$ 。固定半径问题中 $R = R_0 = 0.02$ m；移动边界问题稍后映射到 $0 \leq \xi \leq 1$ 。初始条件为

$$T(r, 0) = 28 \text{ }^\circ\text{C}, \quad C(r, 0) = 2.55 \text{ kg/kg.} \quad (1)$$

这一降维不是把整根药材等同为无空间差异的集中参数系统，而是保留了题目要求的径向分布。其合理性来自圆柱较大的长径比，并将在后文以包含端面的二维 r - z 模型

进行独立检查。

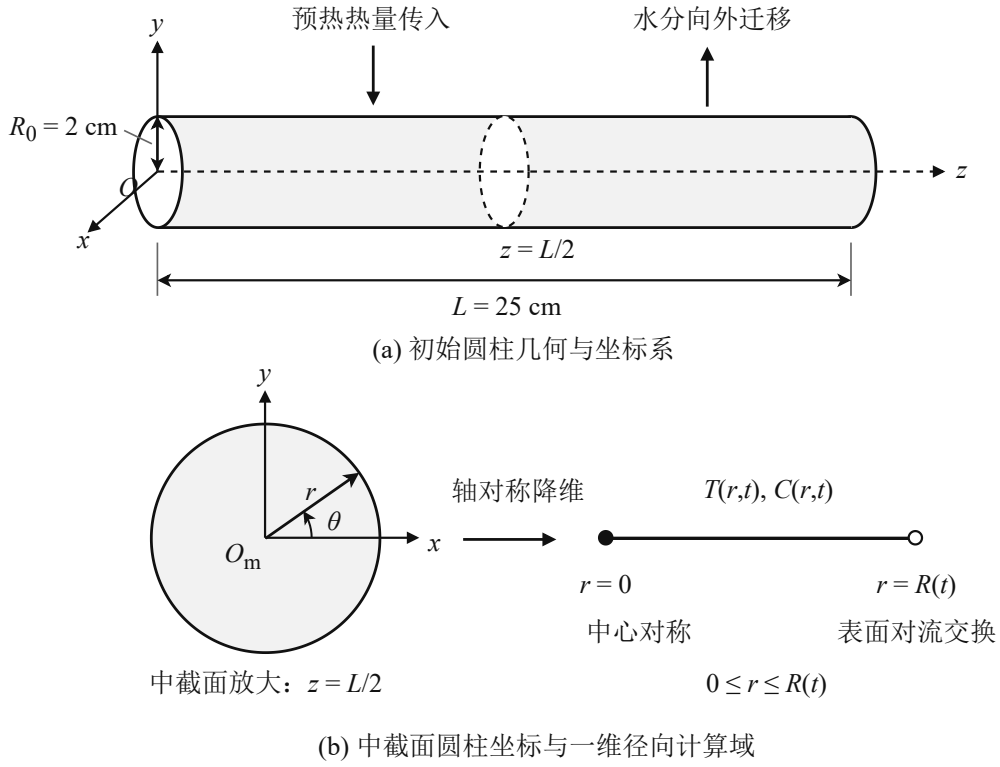


图2 药材初始几何、坐标系与径向计算域（中截面单独放大）

5.2 附件数据检查与边界重构

附件数据的结构与采用方式汇总于表2。附件一、二分别含241个和145个等间隔采样时刻，采样间隔分别为60 s和1800 s，均不同于1 s的数值积分步长，因此需通过插值获得各积分时刻的边界值。

表2 附件时序数据及模型处理方式

数据	覆盖区间	采样间隔	模型处理
环境温度 T_e	0–14400 s	60 s	PCHIP，区间外取 50 °C
环境水分 C_e	0–14400 s	60 s	PCHIP，区间外取 0.05
药材半径 R	0–259200 s	1800 s	单调 PCHIP 并同步求导

对任一相邻数据点 (t_j, y_j) 、 (t_{j+1}, y_{j+1}) ，令 $\vartheta = (t - t_j)/(t_{j+1} - t_j)$ ，PCHIP 在区间内写为

$$y(t) = h_{00}(\vartheta)y_j + h_{10}(\vartheta)\Delta t_j d_j + h_{01}(\vartheta)y_{j+1} + h_{11}(\vartheta)\Delta t_j d_{j+1}, \quad (2)$$

其中

$$h_{00} = 2\vartheta^3 - 3\vartheta^2 + 1, \quad h_{10} = \vartheta^3 - 2\vartheta^2 + \vartheta,$$

$$h_{01} = -2\vartheta^3 + 3\vartheta^2, \quad h_{11} = \vartheta^3 - \vartheta^2.$$

节点斜率 d_j 按相邻割线斜率的保形规则确定, 使单调数据区间不产生新的极值 [3]。相较普通三次样条, 该处理可避免半径在快速收缩段出现非物理回弹; 相较分段线性插值, 其一阶连续性又能稳定给出问题四所需的 $\dot{R}(t)$ 。

附件一的环境温度从 28 °C 上升至末个实测值 50.165 °C, 环境水分浓度从 0.01963 变化至末个实测值 0.04986 kg/kg。实测序列存在小幅波动, 故未强制其单调化; 只有半径数据按物理要求实施单调约束。附件区间内严格采用实测序列的 PCHIP 重构; 最后 1 h 的温度和环境水分均值分别为 49.9989 °C 和 0.049988 kg/kg, 接近所采用的延拓值。据此, 假设超过 14400 s 后环境温度和水分浓度分别保持为 50.000 °C 与 0.05000 kg/kg, 避免把末个采样点的测量波动无限延拓。

5.3 径向热湿控制方程

基于 Fourier 导热定律和 Fick 扩散定律 [1], 固定半径域内的控制方程为

$$\rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left[rk(C)\frac{\partial T}{\partial r}\right], \quad (3)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left[rD(T, C)\frac{\partial C}{\partial r}\right]. \quad (4)$$

式中 ρc_p 是单位体积热容量, 决定温度对净热通量的响应速度; D 是以题设浓度变量定义的等效扩散系数。问题一中除 $D(C)$ 外各物性为常数, 问题二至四中的密度、比热容、导热系数和水分扩散系数均随状态更新。

轴线处没有净通量, 故满足

$$\left.\frac{\partial T}{\partial r}\right|_{r=0} = 0, \quad \left.\frac{\partial C}{\partial r}\right|_{r=0} = 0. \quad (5)$$

侧表面采用“向外法向通量等于表面对流通量”的约定:

$$-k\left.\frac{\partial T}{\partial r}\right|_{r=R} = h_T[T(R, t) - T_e(t)], \quad (6)$$

$$-D\left.\frac{\partial C}{\partial r}\right|_{r=R} = h_m[C(R, t) - C_e(t)]. \quad (7)$$

当环境温度高于表面温度时, (6) 给出指向药材内部的热流; 当表面水分浓度高于环境时, (7) 给出向外水分通量。该符号检验可防止 Robin 条件写反后出现“越烘越湿”等非物理解。

5.4 物性关系及耦合路径

问题一采用题面附录 2 给出的热物性常数与水分扩散系数公式

$$\rho = 820, \quad c_p = 2600, \quad k = 0.36, \quad D = 7.0 \times 10^{-9} \exp(-0.89/C). \quad (8)$$

问题二、三采用题面附录 3 给出的物性关系

$$\begin{aligned} \rho &= 650 + 128C, & c_p &= 1450 + 2736 \frac{C}{C+1}, \\ k &= 0.21 + 0.38 \frac{C}{C+1}, & D &= 2.4 \times 10^{-3} \exp(-0.45/C) \exp[-3850/(T+273.15)]. \end{aligned} \quad (9)$$

问题四采用题面附录 4 给出的物性关系

$$\begin{aligned} \rho &= 760 + 90C, & c_p &= 1850 + 2150 \frac{C}{C+1}, \\ k &= 0.12 + 0.20 \frac{C}{C+1}, & D &= 4.2 \times 10^{-4} \exp(-0.30/C) \exp[-3850/(T+273.15)]. \end{aligned} \quad (10)$$

由 (9) 和 (10) 可知, 水分降低会同时改变储热能力和导热能力, 温度升高则通过 Arrhenius 指数提高 D ; 反过来, 水分场的变化又会改变下一时间层的温度方程系数。本文所称热湿耦合即指这一“水分影响热物性、温度与水分共同影响扩散系数”的闭环, 而非题面未给参数的汽化潜热耦合。

5.5 节点中心有限体积离散

空间节点取 $r_i = R[1 - (1 - i/N)^2]$, $i = 0, 1, \dots, N$, 基准取 $N = 160$, 对低扩散率表层加密; 输出位置仍为每隔 0.1 cm, 由计算解插值得到。内部界面位于 $r_{i+1/2} = (r_i + r_{i+1})/2$ 。按单位轴向长度积分后, 第 i 个环形控制体体积和界面面积分别为

$$V_i = \pi(r_{i+1/2}^2 - r_{i-1/2}^2), \quad A_{i+1/2} = 2\pi r_{i+1/2}. \quad (11)$$

中心控制体下边界直接取 0, 表面控制体上边界取 R 。这种几何构造自动消除了连续方程在 $r = 0$ 处形式上的 $1/r$ 奇点。

以 ϕ 统一表示 T 或 C , 储存系数记为 s , 扩散系数记为 Γ 。在时间层 $n+1$ 采用全隐式格式, 内部节点守恒式为

$$\frac{s_i V_i}{\Delta t} (\phi_i^{n+1} - \phi_i^n) = G_{i+1/2} (\phi_{i+1}^{n+1} - \phi_i^{n+1}) - G_{i-1/2} (\phi_i^{n+1} - \phi_{i-1}^{n+1}), \quad (12)$$

其中

$$G_{i+1/2} = \frac{A_{i+1/2} \Gamma_{i+1/2}}{r_{i+1} - r_i}, \quad \Gamma_{i+1/2} = \frac{\Gamma_i + \Gamma_{i+1}}{2}. \quad (13)$$

温度场取 $s = \rho c_p, \Gamma = k$, 水分场取 $s = 1, \Gamma = D$ 。将 (12) 整理为

$$-a_W \phi_{i-1}^{n+1} + a_P \phi_i^{n+1} - a_E \phi_{i+1}^{n+1} = b_i, \quad (14)$$

其中

$$a_W = G_{i-1/2}, \quad a_E = G_{i+1/2}, \quad a_P = \frac{s_i V_i}{\Delta t} + a_W + a_E, \quad b_i = \frac{s_i V_i}{\Delta t} \phi_i^n.$$

矩阵主对角元为正, 非对角元非正并满足对角占优, 既保留局部守恒, 也有利于保持温度、水分的离散最大值性质 [2]。

中心节点令 $a_W = 0$, 自然实现零通量条件。表面节点将 Robin 通量直接并入控制体:

$$a_P \leftarrow a_P + A_R h, \quad b_N \leftarrow b_N + A_R h \phi_e, \quad A_R = 2\pi R, \quad (15)$$

其中热场取 $(h, \phi_e) = (h_T, T_e)$, 水分场取 (h_m, C_e) 。最终每一场只需求解一个三对角线性系统, 计算量随径向节点数线性增长。

5.6 非线性 Picard 迭代与输出控制

在每个新时间层, 令初始迭代值等于上一时间层结果, 并执行:

1. 由当前 $C^{(m)}$ 计算 ρ, c_p, k , 组装并求解温度三对角系统, 得到 $T^{(m+1)}$;
2. 由 $T^{(m+1)}, C^{(m)}$ 计算 D , 组装并求解水分三对角系统, 得到 $C^{(m+1)}$;
3. 检查

$$\|T^{(m+1)} - T^{(m)}\|_\infty < 10^{-8}, \quad \|C^{(m+1)} - C^{(m)}\|_\infty < 10^{-10};$$

若未同时满足则返回步骤 1, 最多允许 30 次迭代。

四问终算均采用 $\Delta t = 1$ s; 问题二、三共享从初态至达标的同一条轨迹, 问题三按 60 s 抽取, 不再独立积分。程序在每一步检查有限值、 $0 \leq C \leq 2.55$ 、径向单调性、Picard 收敛及方程平衡残差; 阈值跨越时另存插值得到的首达剖面。空间网格、时间步长与输出采样分别检验, 四位小数仅表示结果展示精度。

六、问题一：固定几何与给定热物性的预热模型建立与求解

6.1 恒定热物性预热过程模型

6.1.1 模型特化与特征尺度判断

问题一中半径固定为 $R_0 = 0.02 \text{ m}$ ，密度、比热容和导热系数分别为 820 kg/m^3 、 2600 J/(kgK) 和 0.36 W/(mK) 。因此热扩散率为

$$\alpha = \frac{k}{\rho c_p} = 1.689 \times 10^{-7} \text{ m}^2/\text{s}. \quad (16)$$

初始水分浓度下，由 (8) 得到

$$D_0 = 7.0 \times 10^{-9} \exp(-0.89/2.55) = 4.938 \times 10^{-9} \text{ m}^2/\text{s}. \quad (17)$$

据此可估计径向热、质扩散特征时间

$$\tau_T = \frac{R_0^2}{\alpha} = 2369 \text{ s}, \quad \tau_C = \frac{R_0^2}{D_0} = 8.101 \times 10^4 \text{ s}. \quad (18)$$

水分扩散特征时间约为热扩散特征时间的 34 倍，预示 1800 s 内温度扰动能够明显进入中心，而水分变化主要局限在外层。

进一步计算热、质 Biot 数

$$Bi_T = \frac{h_T R_0}{k} = 1.389, \quad Bi_C = \frac{h_m R_0}{D_0} = 3.240. \quad (19)$$

二者均明显大于集中参数法通常要求的 0.1，说明内部阻力不可忽略。尤其 $Bi_C > Bi_T$ ，表面对流传质并非唯一瓶颈，水分到达表面必须克服更显著的内部扩散阻力。因此，本问不能只解两个平均状态常微分方程，而必须保留径向偏微分方程。

6.2 模型的求解

6.2.1 径向网格与时间步长的选取

表层水分梯度较大，采用向表面加密的径向网格

$$r_i = R_0 \left[1 - \left(1 - \frac{i}{N} \right)^2 \right], \quad i = 0, \dots, N,$$

其中 $N = 160$ ，共 161 个节点。相邻节点中点作为控制体界面，中心与表面界面分别取 0 和 R_0 。按单位轴向长度计算，第 i 个控制体体积为 $V_i = \pi(r_{i+1/2}^2 - r_{i-1/2}^2)$ 。中心控制体呈圆盘形，表面控制体覆盖最外侧半个径向网格区间；该划分可直接处理轴对称边界

而无需引入负半径虚拟点。

取 $\Delta t = 1 \text{ s}$ ，将预热区间 $[0, 1800] \text{ s}$ 划分为 1800 个时间步。各步环境量由附件一的 PCHIP 插值在 t_{n+1} 处求得。计算网格与题目要求的取值位置分开设置，温湿场求出后线性插值至 $0, 0.1, \dots, 2.0 \text{ cm}$ 的 21 个位置。

6.2.2 全隐式控制体方程与边界处理

用 ϕ 统一表示温度或水分，令温度方程的储热系数 $s = \rho c_p$ 、传导系数 $a = k$ ，水分方程对应 $s = 1$ 、 $a = D$ 。相邻节点间的导通系数取

$$G_{i+1/2} = \frac{2\pi r_{i+1/2}}{r_{i+1} - r_i} \frac{a_i + a_{i+1}}{2}.$$

在一个 Picard 迭代内固定物性，内部节点的全隐式方程为

$$-G_{i-1/2}\phi_{i-1}^{n+1} + \left(\frac{s_i V_i}{\Delta t} + G_{i-1/2} + G_{i+1/2} \right) \phi_i^{n+1} - G_{i+1/2}\phi_{i+1}^{n+1} = \frac{s_i V_i}{\Delta t} \phi_i^n. \quad (20)$$

中心内侧通量为零，首行去掉 $G_{-1/2}$ ；表面末行去掉外侧内部通量，并在主对角元加入 $B = 2\pi R_0 h$ 、右端加入 $B\phi_e$ 。其中温度取 $h = h_T$ 、 $\phi_e = T_e$ ，水分取 $h = h_m$ 、 $\phi_e = C_e$ 。所得系统均为三对角矩阵，可用带状线性求解器直接求解。

6.2.3 逐时推进与迭代终止

初始全域取 $T_i^0 = 28 \text{ }^\circ\text{C}$ 、 $C_i^0 = 2.55 \text{ kg/kg}$ 。每个时间步依次执行：

Step 1 初始化：令 $T^{(0)} = T^n$ 、 $C^{(0)} = C^n$ ，确定当前步环境温度和水分浓度。

Step 2 更新温度：将常数 ρ, c_p, k 代入 (20)，施加表面对流边界，求解温度场。

Step 3 更新水分：由当前 $C^{(m)}$ 计算 $D(C^{(m)})$ ，更新界面导通系数，求得 $C^{(m+1)}$ 。各次迭代的右端历史状态始终取上一时间层 C^n 。

Step 4 收敛判断：同时满足 $\|T^{(m+1)} - T^{(m)}\|_\infty < 10^{-8} \text{ }^\circ\text{C}$ 和 $\|C^{(m+1)} - C^{(m)}\|_\infty < 10^{-10} \text{ kg/kg}$ 时接受新时间层，否则继续迭代，最多允许 30 次。

Step 5 推进与结束：检查水分范围、径向单调性及通量平衡后进入下一步；达到 1800 s 时结束预热计算。

本问温度方程为线性，迭代主要用于水分扩散系数的更新；实际各时间层最多 4 次迭代即满足容差。

6.3 结果的分析和验证

6.3.1 温度场计算结果与分析

问题一典型时刻的温度结果见表 3，完整径向曲线见图 3(a)。100 s 时热扰动尚集中在表面附近，中心温度只比初值高 $6.4 \times 10^{-5} \text{ }^{\circ}\text{C}$ ；随着热量持续输入，600 s 后整个截面开始明显升温。到 1800 s，中心与表面分别达到 $33.5770 \text{ }^{\circ}\text{C}$ 和 $36.7865 \text{ }^{\circ}\text{C}$ 。

表 3 问题一典型时刻温度分布 ($^{\circ}\text{C}$)

t/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	28.0001	28.0004	28.0041	28.0324	28.1802
300	28.0414	28.0642	28.1522	28.3683	28.8495
600	28.4549	28.5377	28.8053	29.3168	30.1650
900	29.3262	29.4602	29.8774	30.6177	31.7305
1200	30.5447	30.7118	31.2241	32.1143	33.4304
1500	31.9976	32.1887	32.7681	33.7486	35.1218
1800	33.5770	33.7736	34.3654	35.3631	36.7865

温度剖面由中心向表面平滑升高，1800 s 时表面—中心温差为 $3.2095 \text{ }^{\circ}\text{C}$ ，与中心升温滞后的预期一致。

6.3.2 水分场计算结果与分析

典型时刻水分浓度列于表 4，其空间曲线见图 3(b)。100 s 时表面水分已由 2.55 降至 2.2475 kg/kg ，而 $r \leq 1.0 \text{ cm}$ 区域在四位小数精度下仍保持初值。到 1800 s，表面进一步降至 1.5104 kg/kg ，中心仍为 2.5500 kg/kg ，形成 1.0396 kg/kg 的中心—表面差。

表 4 问题一典型时刻水分浓度 (kg/kg)

t/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	2.5500	2.5500	2.5500	2.5500	2.2475
300	2.5500	2.5500	2.5500	2.5492	2.0511
600	2.5500	2.5500	2.5500	2.5352	1.8772
900	2.5500	2.5500	2.5497	2.5045	1.7549
1200	2.5500	2.5500	2.5482	2.4646	1.6587
1500	2.5500	2.5499	2.5445	2.4206	1.5789
1800	2.5500	2.5497	2.5383	2.3755	1.5104

温度和水分的中心—表面差见表 5，预热阶段两者均增大。

表 5 问题一中心—表面差的演化

t/s	$T(R, t) - T(0, t)/^{\circ}\text{C}$	$C(0, t) - C(R, t)/(\text{kg/kg})$
100	0.1801	0.3025
300	0.8081	0.4989
600	1.7101	0.6728
900	2.4043	0.7951
1200	2.8857	0.8913
1500	3.1242	0.9711
1800	3.2095	1.0396

表层失水使 $D(C)$ 降低，内部水分迁移较慢，因此表面水分不能代表中心状态。

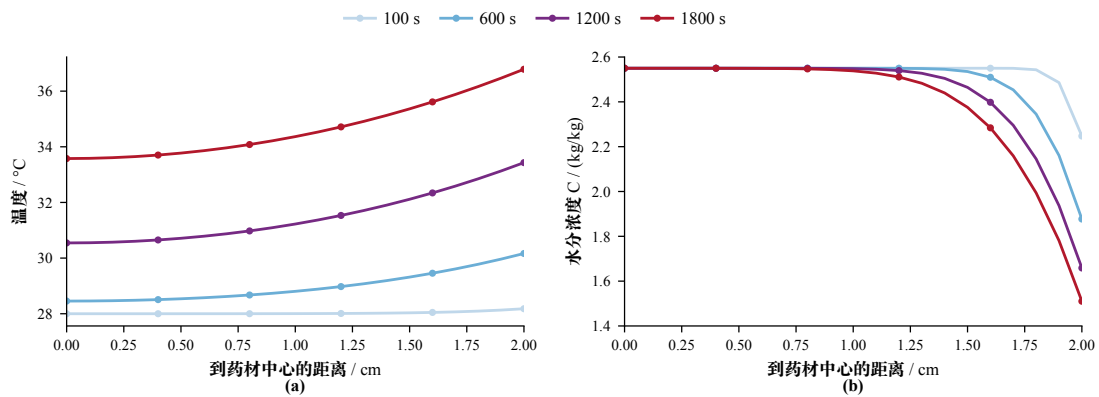


图 3 问题一温度与水分径向剖面

6.3.3 预热温度的连续时空演化

图 4展示 0–30 min 的温度时空分布及等温线，热量由表面逐渐传向中心。

6.3.4 二维轴对称检验

为检验端面传递对中截面结果的影响，另建立半长圆柱轴对称有限体积模型。检验坐标取 $z = 0$ 为中截面、 $z = L/2$ 为端面，轴线与中截面零通量，侧表面和端面均施加热、质 Robin 边界。在 41 个径向、21 个半轴向节点及 10 s 步长下计算至 1800 s，与配对一维结果比较，全时段中截面最大温差为 $7.49 \times 10^{-7} ^{\circ}\text{C}$ 、最大水分差为 $7.35 \times 10^{-14} \text{ kg/kg}$ ，支持本问预热阶段的中截面径向简化。

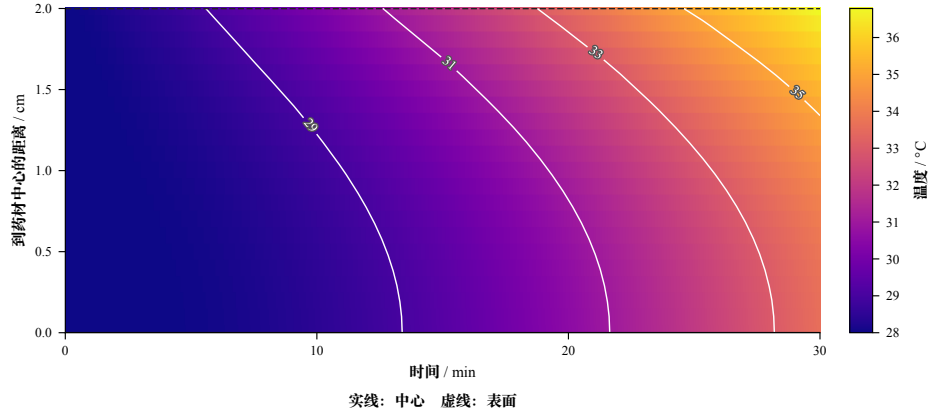


图 4 问题一预热温度的时空分布及等温线 (0–30 min)

6.3.5 本问结论与分析

预热 1800 s 后中心显著升温，但水分仍接近初值；数值剖面满足中心对称、温度内低外高和水分内高外低的特征。

七、问题二：变物性热湿迁移模型的建立与求解

7.1 变物性热湿迁移模型

7.1.1 非线性模型及耦合关系

问题二保持 $R = 0.02 \text{ m}$ 、 $h_T = 25 \text{ W}/(\text{m}^2\text{K})$ 和 $h_m = 8 \times 10^{-7} \text{ m/s}$ 不变，将 (9) 代入 (3) 和 (4)。此时温度方程的储存项和扩散项都依赖 C ，水分方程的扩散系数同时依赖 T 和 C 。若写成算子形式，可表示为

$$\mathcal{H}[T; C] = 0, \quad \mathcal{M}[C; T] = 0, \quad (21)$$

其中两个算子通过状态相关系数双向联系。该耦合不要求在一个大矩阵中同时求解全部未知量：在每次 Picard 迭代内，先固定 C 解 $\mathcal{H}$ ，再用更新后的 T 解 $\mathcal{M}$ ，即可逐步逼近同一时间层的自洽状态。

与问题一相比，问题二初始状态下的物性参数与题面附录 2 给出的常数不同。将 $T = 28^\circ\text{C}$ 、 $C = 2.55 \text{ kg/kg}$ 代入题面附录 3 的物性关系，得到初始密度 976.4 kg/m^3 、比热容 $3415.3 \text{ J}/(\text{kgK})$ 、导热系数 $0.4830 \text{ W}/(\text{mK})$ 和扩散系数 $5.642 \times 10^{-9} \text{ m}^2/\text{s}$ 。因此，两问在相同几何和表面对流系数下仍会呈现不同响应，不能把问题一的 1800 s 结果直接作为问题二的初段结果。

7.1.2 物性随状态的演化

为展示非线性系数的实际变化，将初态以及 3 h 的中心、表面状态代入 (9)，结果见表 6。随着水分降低， ρ, c_p, k 均减小；这意味着单位体积储热能力降低，同时内部导热能力也有所下降。二者对升温速度具有相反影响，需要由完整能量方程共同决定。

表 6 问题二代表状态下的物性参数

状态	$\rho/(\text{kg/m}^3)$	$c_p/[\text{J}/(\text{kg K})]$	$k/[\text{W}/(\text{m K})]$	$D/(\text{m}^2/\text{s})$
初始全域	976.4	3415.3	0.4830	5.642×10^{-9}
3 h 中心	876.1	3196.9	0.4526	1.239×10^{-8}
3 h 表面	779.0	2823.5	0.4008	1.027×10^{-8}

扩散系数同时受温度和水分影响。升温使 Arrhenius 因子 $\exp[-3850/(T + 273.15)]$ 增大，促进水分扩散；干燥使 $\exp(-0.45/C)$ 减小，抑制扩散。3 h 时两种效应叠加后，中心和表面 D 都高于初值；尽管表面温度略高，其扩散系数 D 却小于中心值。这说明低含水表层已开始形成扩散阻力，水分迁移并非随温度升高而无限加速。

7.2 模型的求解

7.2.1 计算网格与非线性系数更新

沿用问题一的 161 个表面加密节点和 1 s 时间步，但重新从 $T_i^0 = 28^\circ\text{C}$ 、 $C_i^0 = 2.55 \text{ kg/kg}$ 出发，采用题面附录 3 的物性公式计算，不能以问题一的预热结果代替初段解。前 3 h 用于本问温湿场分析，随后沿同一轨迹继续积分，为问题三确定达标时间。

对每个时间层，以 $T^{(0)} = T^n, C^{(0)} = C^n$ 为迭代初值。第 m 次迭代先由 $C^{(m)}$ 更新 ρ, c_p, k ，再以 $s_i^{(m)} = \rho_i^{(m)} c_{p,i}^{(m)}$ 和 $k_i^{(m)}$ 组装温度方程。界面系数采用相邻节点算术平均，中心零通量与表面对流条件按问题一的方法进入矩阵。

7.2.2 温湿场交替求解

将 (20) 分别用于温度、水分，记对应三对角矩阵为 A_T, A_C 。一次交替迭代可写为

$$A_T(C^{(m)})T^{(m+1)} = \text{diag}\left(\frac{\rho^{(m)}c_p^{(m)}V}{\Delta t}\right)T^n + b_T,$$

$$A_C(T^{(m+1)}, C^{(m)})C^{(m+1)} = \text{diag}\left(\frac{V}{\Delta t}\right)C^n + b_C.$$

其中 b_T, b_C 为表面环境交换项。水分方程使用新温度与当前水分估计更新扩散系数，温度相关指数统一采用开尔文温度。两式右端的历史状态分别保持为 T^n, C^n ，只在迭代收

收敛后推进时间。

具体步骤如下：

Step 1 更新边界：在 t_{n+1} 读取 PCHIP 环境值；超过 14400 s 后采用 50 °C、0.05 kg/kg 的长期边界。

Step 2 求解温度：按当前水分估计更新储热系数和导热系数，求解温度三对角系统。

Step 3 求解水分：以新温度和当前水分估计更新扩散系数，求解水分三对角系统。

Step 4 判断收敛：采用问题一的温度 10^{-8} 、水分 10^{-10} 双重无穷范数容差；未满足时返回物性更新，满足后接受该时间层。

7.2.3 步长选取与计算检查

全隐式处理避免显式扩散格式对最小网格宽度的严格步长限制，但步长仍影响时间精度。采用 1 s 可解析附件一 60 s 采样间隔内的边界变化；长期达标时间的 2 s 与 1 s 对照相对差为 1.52×10^{-5} ，网格检验见后文。每步检查状态有限性、 $0 \leq C \leq 2.55$ 和径向单调性，并将控制体累积变化与表面通量核对。实际最多 5 次 Picard 迭代即收敛，未触及 30 次上限。

7.3 结果的分析 and 验证

7.3.1 温度结果及其空间演化

典型时刻温度见表 7，空间曲线见图 5(a)。0.5 h 时中心和表面分别为 32.1911 °C、35.4141 °C，仍有 3.2230 °C 的明显差异；1.5 h 时，温差降为 1.2928 °C；到 3 h 仅剩 0.1173 °C。这表明温度先经历表面快速升温 and 内部滞后，随后在环境逐渐稳定时趋于径向均匀。

表 7 问题二温度分布 (°C)

t/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5000	32.1911	32.3840	32.9674	33.9620	35.4141
1	40.3822	40.5543	41.0609	41.8784	43.0001
1.5000	45.8464	45.9345	46.1927	46.6049	47.1393
2	48.4498	48.4877	48.5980	48.7732	49.0030
2.5000	49.4667	49.4790	49.5133	49.5645	49.6617
3	49.8493	49.8552	49.8746	49.9100	49.9666

7.3.2 水分结果及控制机制

水分结果见表 8和图 5(b)。0.5 h 时中心仍为 2.5499 kg/kg，表面已降至 1.6487 kg/kg；3 h 时中心和表面分别降至 1.7662 和 1.0081 kg/kg。中心—表面差由 0.5 h 的 0.9012 降至 3 h 的 0.7581 kg/kg，说明水分扰动已经传播到中心，但内部尚未达到近似均匀状态。

表 8 问题二水分浓度分布 (kg/kg)

t/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5000	2.5499	2.5489	2.5255	2.3257	1.6487
1	2.5257	2.4947	2.3578	2.0230	1.4712
1.5000	2.3860	2.3256	2.1344	1.8020	1.3476
2	2.1708	2.1084	1.9236	1.6260	1.2311
2.5000	1.9567	1.9006	1.7360	1.4720	1.1167
3	1.7662	1.7165	1.5702	1.3333	1.0081

3 h 时温度已接近均匀，而中心水分仍为表面的约 1.75 倍，说明温度趋于均匀不能作为烘干结束的依据。

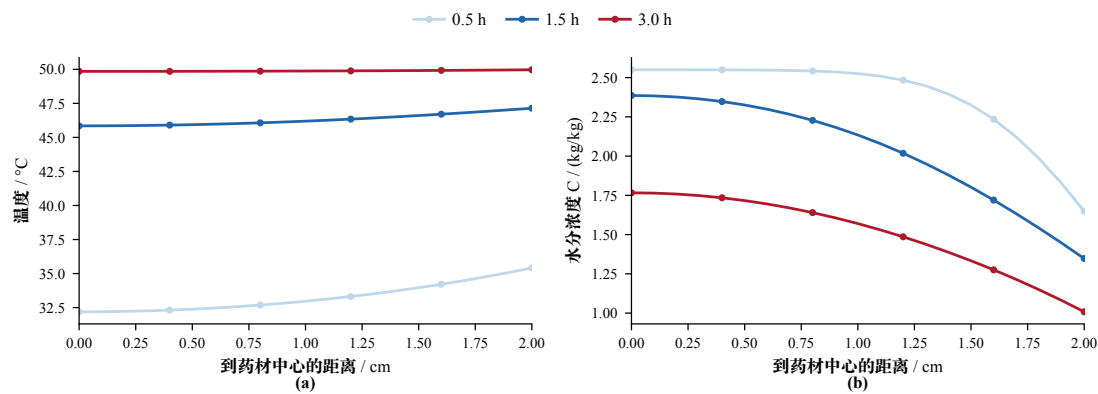


图 5 问题二变物性温度与水分剖面

7.3.3 水分迁移与扩散系数的时空证据

图 6展示前 3 h 水分时空分布，失水影响由表面向中心传播。

图 7展示 $D(T, C)$ 的时空变化：升温增强扩散，但表层失水使其扩散系数低于内部。

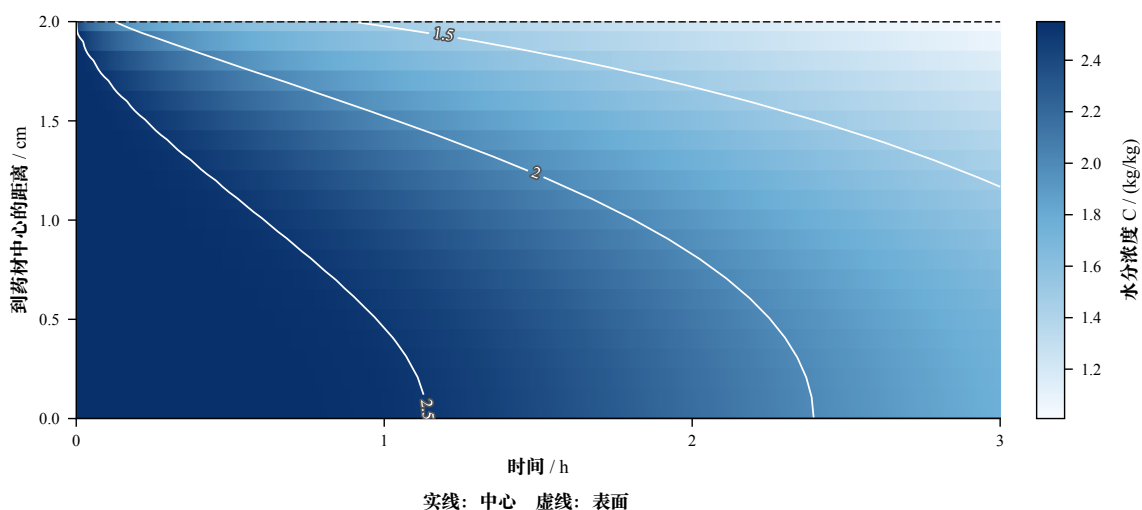


图 6 问题二前 3 h 水分浓度的时空分布及等浓度线

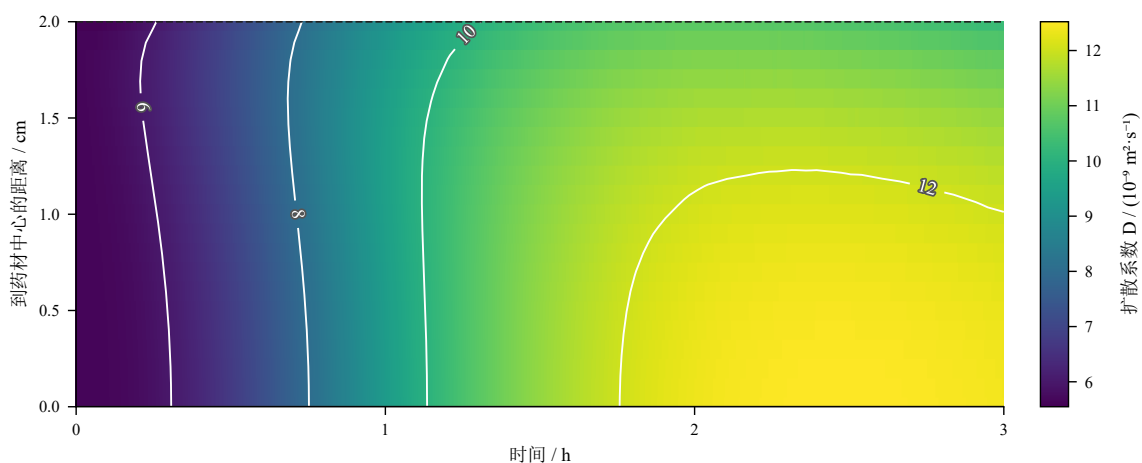


图 7 问题二状态相关扩散系数的时空云图及等值线

7.3.4 共享轨迹一致性检验

问题二、三采用相同物性、网格和边界连续计算。第二问逐秒温湿场按 60 s 抽取后，与第三问全部共同时间层逐元素一致，最大差为零；长期网格与时间步检验见问题三。这一检查确认短期温湿响应与长期达标预测采用相同数值轨迹。

7.3.5 本问结论与分析

各时间层最多 5 次 Picard 迭代即收敛，温湿剖面保持平滑。3 h 时中心—表面温差为 0.1173 °C、水分差为 0.7581 kg/kg，后续干燥由内部水分迁移控制。

八、问题三：全截面达标时间模型的建立与求解

8.1 全截面达标时间模型

8.1.1 全域首达判据

问题三要求“所有位置”的水分浓度均不超过 0.15 kg/kg，因此不能用截面平均值或表面值代替。定义空间最大水分浓度

$$M(t) = \max_{0 \leq r \leq R_0} C(r, t), \quad (22)$$

则烘干完成时刻是集合约束的首次满足时间

$$t_d = \inf\{t \geq 0 : M(t) \leq 0.15\}. \quad (23)$$

该定义包含两个必要条件：一是 t_d 时全空间均达标，二是在任何 $t < t_d$ 时至少存在一个未达标位置。因而它比“取第一个离散输出时刻”更严格，也能避免输出间隔改变时答案随之改变。

初始水分场均匀，表面环境水分始终显著低于药材内部，且有限体积矩阵是对角占优的 M 型结构。在这种向外失水的条件下，数值解应保持由中心向表面非增。程序对每一个内部时间层检查

$$C_i^n - C_{i+1}^n \geq -2 \times 10^{-9}, \quad i = 0, 1, \dots, N-1, \quad (24)$$

全部时间层均通过。因此

$$M(t) = C(0, t), \quad (25)$$

即轴线是本问的控制位置。这里并非预先假定中心最大，而是先保留全域最大值判据，再由计算结果确认其可化为中心事件。

8.2 模型的求解

8.2.1 长期积分与首达区间搜索

沿用问题二的网格、1 s 步长及题面附录 3 的物性，从初始状态连续推进，不在 3 h 处重新初始化。附件一时间范围内采用 PCHIP 环境值，超过 4 h 后取 $T_e = 50^\circ\text{C}$ 、 $C_e = 0.05 \text{ kg/kg}$ 。每步先完成温湿场 Picard 迭代，再计算全部径向节点中的最大水分 M^{n+1} 。

Step 1 初始化判据：令 $t_0 = 0$ 、 $M^0 = 2.55$ ，标记尚未达到阈值。

Step 2 推进状态：按问题二交替求解温度 and 水分，并检查 (24)。

Step 3 寻找跨越: 若 $M^{n+1} > 0.15$, 令 $n \leftarrow n + 1$ 继续积分; 第一次出现 $M^n > 0.15$ 且 $M^{n+1} \leq 0.15$ 时, 确定首达区间 $[t_n, t_{n+1}]$ 。

搜索使用全域最大值, 中心控制仅在径向单调性检验通过后得到确认。如此可同时满足“全域均达标”与“首次达标”的要求。

8.2.2 阈值事件定位方法

长期积分使用 $\Delta t = 1$ s。设第 n 层尚未达标而第 $n + 1$ 层已经达标, 即

$$M^n > 0.15, \quad M^{n+1} \leq 0.15.$$

在这两个相邻时间层间定义

$$\lambda = \frac{0.15 - M^n}{M^{n+1} - M^n}, \quad t_d = t_n + \lambda \Delta t, \quad (26)$$

并同步插值得到达标剖面

$$C(r, t_d) = C^n(r) + \lambda[C^{n+1}(r) - C^n(r)]. \quad (27)$$

由于 $0 \leq \lambda \leq 1$, 所得时刻位于数值解跨越阈值的 1 s 区间内。时间精度由积分步长和事件插值决定, 题目要求的 60 s 取值间隔只用于结果展示。

8.2.3 达标剖面与数值精度核验

确定 t_d 后, 使用同一插值权重计算温度、水分剖面, 核对最大水分等于阈值且其余位置不超过阈值。每 60 s 的径向分布由同一连续轨迹取值; 若首达时间不是整分钟, 则继续推进到其后的首个整分钟, 以覆盖题目要求的时间序列。

终算采用 161 节点、1 s 步长, 网格与时间步加密对照见表 10。

8.3 结果的分析 and 验证

8.3.1 干燥时间与径向分布

由 (26) 得到

$$t_d = 206898.8710 \text{ s} = 57.4719 \text{ h}. \quad (28)$$

典型时刻及达标瞬间的水分浓度见表 9。中心从 6 h 的 1.0170 降至 12 h 的 0.4562, 随后下降速度逐渐减慢; 达标时中心恰为 0.1500, 而表面已降至 0.0527 kg/kg, 接近稳定环境水分浓度。

表 9 问题三每 6 h 及达标时的水分浓度 (kg/kg)

t/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4562	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2916	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0549
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
57.4719	0.1500	0.1477	0.1402	0.1249	0.0527

中心、表面轨迹及径向剖面见图 8。左图用水平虚线标出全域达标阈值，右图给出 6、18、36、48 h 及达标瞬间的径向剖面。表面于 12.5525 h 达标，比中心提前 44.9194 h，应以内部最大水分决定停机时刻。

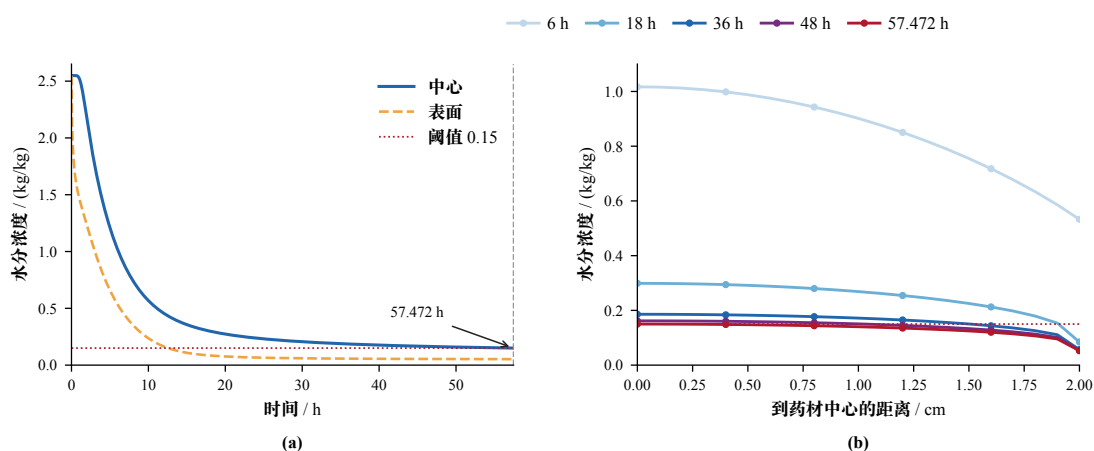


图 8 问题三中心、表面水分轨迹及典型时刻径向分布

低含水状态下扩散系数减小，后期干燥明显变慢：48 h 中心水分为 0.1618，仍需约 9.47 h 才能达到阈值。

8.3.2 达标区域由表面向中心的推进

图 9 显示已达标区域由表面向中心推进，未达标核心最终于 57.472 h 消失。红线为浓度阈值等值线，不是相变界面。

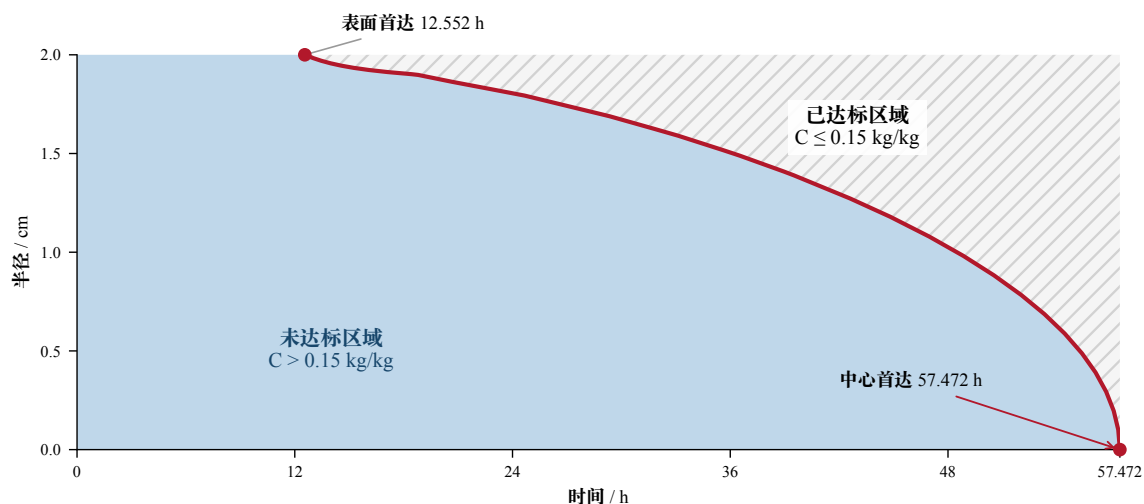


图9 问题三已达标区域与未达标区域的演化；红线为含水率阈值边界

8.3.3 空间网格与时间步长检验

先固定 10 s 步长，比较 41、81、161、321 个径向节点，再固定 161 个节点将步长由 2 s 减为 1 s，结果见表 10。空间与时间加密的达标时间相对变化分别为 1.65×10^{-5} 、 1.52×10^{-5} ，均小于预设空间 10^{-3} 、时间 10^{-4} 的相对容差。这些差值表示离散加密的经验变化量，不是严格误差上界。

表 10 问题三网格与步长收敛检验：达标时间 (h)

计算设置	问题三
41 节点, 10 s	57.461591
81 节点, 10 s	57.476003
161 节点, 10 s	57.479757
321 节点, 10 s	57.480706
161 节点, 2 s	57.472781
161 节点, 1 s (终算)	57.471909

8.3.4 端面影响的二维检验

沿用问题一的半长圆柱二维边界设置，将计算延伸至全二维区域达标，统一采用 30 s 步长，并与相同径向网格和步长的一维计算配对。41×21 节点下，二维达标时间为 57.476411 h，配对相对差为 4.55×10^{-5} ；81×41 节点下为 57.491233 h，相对差为 3.84×10^{-5} 。两种网格的最大水分均位于轴线与中截面交点。

41×21 网格下，中截面最大温差为 0.001775 °C，最大水分差为 2.41×10^{-5} kg/kg；端面与中截面最大水分差约为 0.970 kg/kg。因此中部径向模型能够描述控制达标的位

置，但不能代替端面局部状态。二维检验步长与 1 s 终算不同，其时长不用于替换主模型答案。

8.3.5 环境边界灵敏度分析

保持前 4 h 实测边界不变，对后续 $T_{e,\infty} = 50\text{ }^{\circ}\text{C}$ 施加 $\pm 2.5\text{ }^{\circ}\text{C}$ 扰动，对 $C_{e,\infty} = 0.05\text{ kg/kg}$ 施加 $\pm 5\%$ 扰动。所有方案采用 161 节点、10 s 步长，结果见表 11。

表 11 问题三环境边界扰动下的干燥时长 (h)

情形	问题三
基准 (10 s)	57.4798
恒温阶段温度 $-2.5\text{ }^{\circ}\text{C}$	62.3098
恒温阶段温度 $+2.5\text{ }^{\circ}\text{C}$	53.1605
恒温阶段环境水分 -5%	57.4320
恒温阶段环境水分 $+5\%$	57.5322

用温度对称差分导数与水分弹性衡量影响：

$$K_T = \frac{t_d(T_e + \delta T) - t_d(T_e - \delta T)}{2\delta T}, \quad S_C = \frac{t_d(1.05C_e) - t_d(0.95C_e)}{0.10t_d(C_e)}. \quad (29)$$

得到 $K_T = -1.8299\text{ h}/^{\circ}\text{C}$ 、 $S_C = 0.0174$ ；以开尔文温度定义 $S_T = (T_{e,K}/t_d)K_T$ ，得 $S_T = -10.2874$ 。在所设扰动下，升温缩短干燥时间，提高环境水分则延长时间，且温度影响更明显。

8.3.6 本问结论与分析

时间步与空间网格加密所得达标时间相对差均小于 2×10^{-5} ，全部剖面通过径向单调性检查，支持上述首达时间的数值稳定性。

九、问题四：收缩移动边界模型的建立与求解

9.1 收缩移动边界模型

9.1.1 半径数据重构与固定域映射

附件二给出的半径由初始 2 cm 持续减小至 1.2 cm。经单调 PCHIP 重构后，在本问主要时刻的半径见表 12。前 6 h 半径已经缩小到 1.374 cm，完成总收缩量的 78.3%；此后收缩速度逐渐减小，从 36 h 至本问达标时刻，重构半径保持为 1.200 cm。由此可见，几何变化主要集中在干燥早期，但它对随后全部扩散过程都会持续产生影响。

表 12 问题四典型时刻的重构半径

t/h	0	6	12	18	24	30	36	48	51.088
R/cm	2.000	1.374	1.248	1.214	1.204	1.201	1.200	1.200	1.200

若直接在物理域 $0 \leq r \leq R(t)$ 上使用固定的 0.1 cm 网格, 节点数会随时间减少, 旧节点场量还需频繁插值到新网格, 不利于保持守恒。为此定义无量纲固定域坐标

$$\xi = \frac{r}{R(t)}, \quad 0 \leq \xi \leq 1, \quad (30)$$

并令 $\psi(\xi, t) = \phi(r, t)$ 。半径收缩时, 固定 ξ 的节点随材料截面同步向内移动, 始终保留 161 个表面加密计算节点, 不需要改变矩阵维数。

9.1.2 收缩模型方程

本问采用均匀径向收缩假设, 材料速度与材料坐标网格速度相同。令

$$\xi = \frac{r}{R(t)}, \quad 0 \leq \xi \leq 1, \quad (31)$$

则相对输运速度为零, 温度和水分子方程均可写成材料坐标下的扩散方程

$$\frac{\partial \psi}{\partial t} = \frac{1}{R^2 \xi} \frac{\partial}{\partial \xi} \left(\xi \Gamma \frac{\partial \psi}{\partial \xi} \right), \quad (32)$$

水分取 $\psi = C, \Gamma = D$, 温度取 $\psi = T, \Gamma = k$, 温度方程左侧采用题给有效储热系数 ρc_p 。因此, 收缩效应主要通过内部扩散项的 R^{-2} 和表面交换项的 R^{-1} 进入模型。收缩干燥的物质平衡需明确内部速度与边界通量 [5]; 均匀收缩是本文的建模假设, 实际材料的局部变形和裂纹未在本模型中考虑。

表面边界也必须同步变换。由 $\partial/\partial r = R^{-1} \partial/\partial \xi$ 得

$$-\frac{k}{R} \frac{\partial T}{\partial \xi} \Big|_{\xi=1} = h_T (T_s - T_e), \quad (33)$$

$$-\frac{D}{R} \frac{\partial C}{\partial \xi} \Big|_{\xi=1} = h_m (C_s - C_e). \quad (34)$$

因此在无量纲控制体方程中, 内部扩散导通系数乘 R^{-2} , 表面对流导通系数乘 R^{-1} 。若只缩放网格而遗漏这些尺度因子, 模型将不能正确反映半径变化带来的通量放大。

9.2 模型的求解

9.2.1 材料坐标离散与迭代流程

在 $\xi_i = 1 - (1 - i/160)^2$ 上采用 161 个表面加密节点，时间步长取 1 s。每个时间层先由 PCHIP 更新半径和环境边界，再更新 R^{-2} 内部扩散项及 R^{-1} 表面交换项；随后采用 Picard 迭代依次求解温度场和水分场。全域最大水分浓度首次低于 0.15 kg/kg 时停止，并对相邻时间层线性插值定位达标时刻。计算流程见图 10。

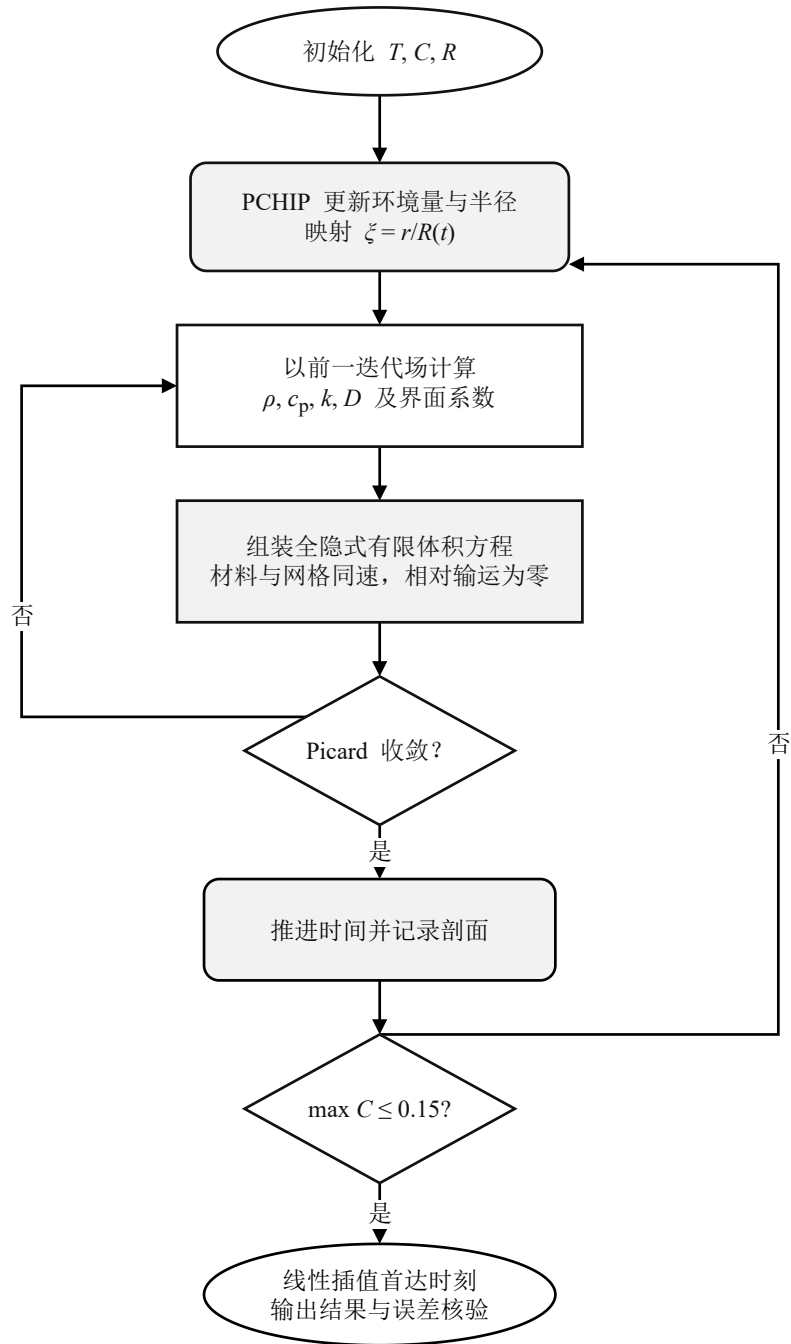


图 10 问题四移动边界全隐式迭代流程

为分离几何收缩作用，另保持题面附录 4 的物性和环境不变，将半径固定为 2 cm，

按相同步长与首达判据求解对照方案。

9.2.2 物理位置输出规则

计算解首先定义在固定材料坐标 ξ 上，而题目模板给出的列对应固定物理位置 $r_j = 0, 0.1, \dots, 1.9$ cm。对任一时刻，仅当 $r_j \leq R(t)$ 时计算

$$C(r_j, t) = \mathcal{I}_\xi[C(\xi, t)]|_{\xi=r_j/R(t)}, \quad (35)$$

其中 $\mathcal{I}_\xi$ 直接使用全部 161 个计算节点作线性插值；当 $r_j > R(t)$ 时，该位置已经处于药材外部，单元格留空而不是填 0。最后一列始终记录 $C(\xi = 1, t)$ ，即随半径移动的实时表面。

该规则区分固定物理位置和随材料移动的表面位置，避免将药材外部的环境量误认为材料状态。

9.2.3 坐标实现的交叉验证

为检验固定材料坐标实现的可靠性，另在物理坐标中建立移动控制体模型。两种方法使用相同物性、环境、时间步长和首达判据独立计算，所得水分轨迹和达标时间在图示精度下重合。该检验针对坐标离散实现的一致性，详细对照见表 15 and 图 14。

9.3 结果的分析 and 验证

9.3.1 达标时间与浓度分布

问题四继续以全域最大水分浓度作为终止量。全部剖面保持中心最大，故事件定位仍可用 (26) 和 (27)。计算得到

$$t_d = 183915.7175 \text{ s} = 51.0877 \text{ h}, \quad R(t_d) = 1.2000 \text{ cm}. \quad (36)$$

典型时刻的固定物理位置与动态表面水分见表 13。6 h 时中心水分仍为 1.7188，动态表面已降至 0.4205 kg/kg；24 h 时中心为 0.2851，表面只有 0.0673；达标时中心为 0.1500，表面为 0.0526。

动态表面在 12.7848 h 已达到 0.15，而中心需要 51.0877 h，两者仍相差 38.3029 h。收缩虽然显著缩短了内部扩散距离，却没有消除“外干内湿”的过程特征，因此实际控制仍应关注中心或内部代理量。

表 13 问题四典型时刻水分浓度 (kg/kg)

t/h	0 cm	0.5 cm	1.0 cm	动态表面
6	1.7188	1.5370	1.0221	0.4205
12	0.7374	0.6544	0.4082	0.1669
18	0.4087	0.3686	0.2397	0.0892
24	0.2851	0.2610	0.1790	0.0673
30	0.2263	0.2094	0.1492	0.0595
36	0.1928	0.1796	0.1316	0.0560
42	0.1712	0.1604	0.1200	0.0541
48	0.1561	0.1468	0.1116	0.0530
51.0877	0.1500	0.1413	0.1081	0.0526

9.3.2 收缩效应的反事实分解

问题三和问题四的物性函数不同，直接比较 57.4719 h 与 51.0877 h 不能将全部差异归因于收缩。为保持物性和环境相同，另作两组对照：

1. 使用题面附录 4 的物性和相同环境，但保持 $R = 2$ cm，得到固定半径反事实时长 129.8445 h；
2. 保留真实 $R(t)$ ，采用原固定空间标量方程，在 ξ 坐标下保留系数为 $-\dot{R}\xi/R$ 的对流项，得到 52.6375 h，仅用作模型结构比较。

第一组在本文材料收缩闭合下给出

$$\Delta t_R = 129.8445 - 51.0877 = 78.7568 \text{ h}, \quad \eta_R = \frac{\Delta t_R}{129.8445} = 60.65\%. \quad (37)$$

第二组与材料模型相差 1.5498 h，表明速度描述影响长期预测。该差异不能解释为独立“坐标移动效应”，也不能据此认定任一方程已被实验验证。固定半径与均匀收缩模型的中心水分轨迹及半径变化见图 11。

9.3.3 收缩材料域与截面水分的联合展示

图 12(a) 给出收缩边界与水分场；(b) 为径向解的轴对称截面重建，各截面共用长度比例和色标。几何收缩先于深部水分达标，重建截面不属于独立二维计算。

9.3.4 COMSOL 有限元复核

COMSOL 采用相同物性、环境与半径历史，并设置轴向均匀、端面绝热。在 6、12、24、36、48 h 及主模型首达时刻，分别取 1001 个共同径向位置，将有限元解与由原始 161

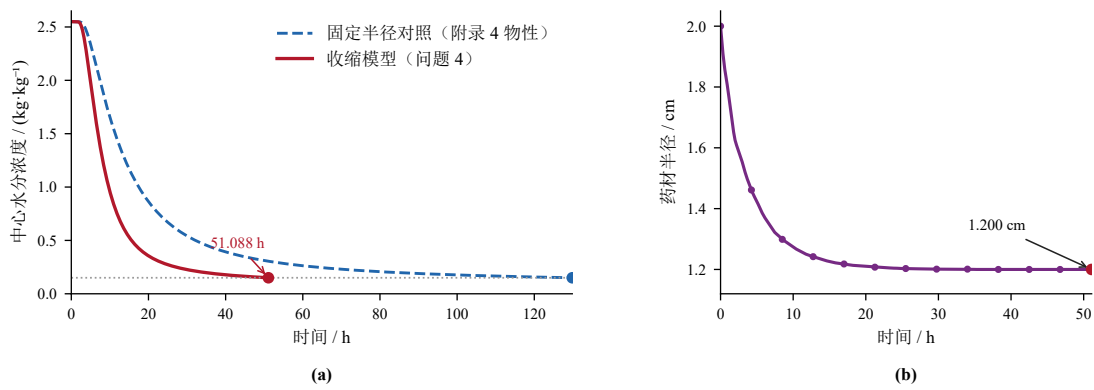


图 11 问题四收缩与固定半径反事实比较

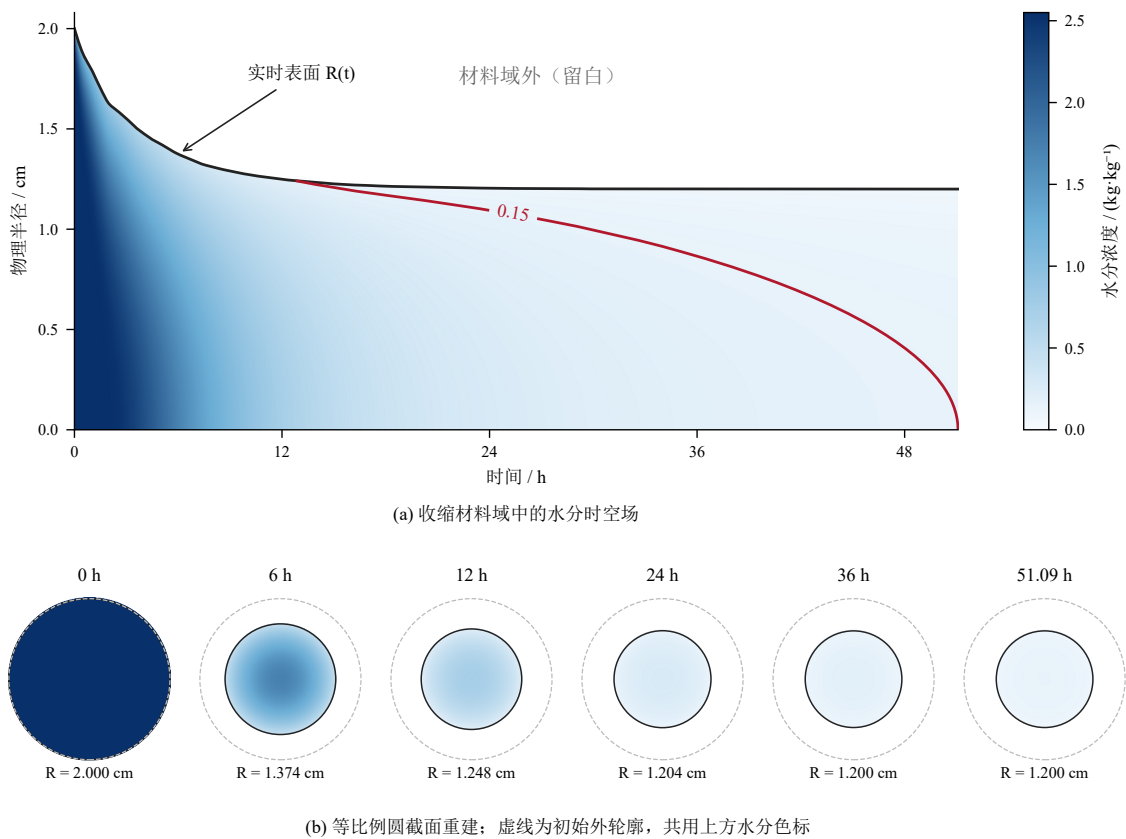


图 12 问题四收缩材料域的水分时空场及等比例截面重建

节点线性插值得到的有限体积解比较。表 14 统一列出这一采样口径下的网格与容差检验，表中加密单元数指径向单元数。初始自动网格下最大水分差约为 $1.17 \times 10^{-2} \text{ kg/kg}$ ；径向加密至 320 单元、相对求解容差收紧至 10^{-7} 后，六个对照时刻的最大水分差为 $4.54 \times 10^{-5} \text{ kg/kg}$ 。图 13 采用该加密设置，展示径向剖面、采样点对照及半径历程。主模型首达时刻 51.0877 h 对应的 COMSOL 中心水分 0.14999888 kg/kg ，该值不是 COMSOL 独立首达时间。上述检验支持既定模型的数值一致性，不替代对物理假设的实验验证。

表 14 COMSOL 网格与求解容差检验

网格设置	相对求解容差	最大水分差 (kg/kg)
初始自动网格	1.00×10^{-4}	1.17×10^{-2}
初始自动网格, 收紧容差	1.00×10^{-6}	1.17×10^{-2}
表层加密, 80 单元	1.00×10^{-6}	4.52×10^{-5}
表层加密, 160 单元	1.00×10^{-6}	4.53×10^{-5}
表层加密, 320 单元	1.00×10^{-6}	4.19×10^{-5}
320 单元, 收紧容差	1.00×10^{-7}	4.54×10^{-5}

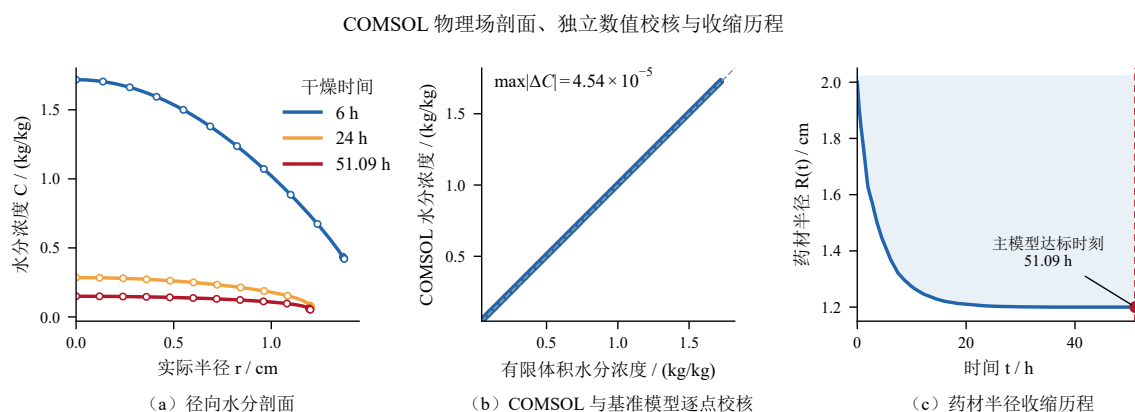


图 13 加密 COMSOL 有限元与有限体积解的剖面对照及半径收缩历程

9.3.5 移动网格独立验证

按前述移动控制体方法独立计算，比较结果见表 15 和图 14。

表 15 移动网格与固定域映射的数值比较

指标	固定域映射法	移动网格法	相对差
t_d/h	51.087699300	51.087699300	5.81×10^{-13}
6 h, 中心	1.718798385	1.718798385	1.68×10^{-14}
6 h, 动态表面	0.420513845	0.420513845	2.92×10^{-13}
24 h, 中心	0.285074422	0.285074422	1.29×10^{-13}
24 h, 动态表面	0.067330674	0.067330674	4.91×10^{-14}
48 h, 中心	0.156140474	0.156140474	2.83×10^{-13}
48 h, 动态表面	0.053014141	0.053014141	1.11×10^{-13}
t_d^{map} , 中心	0.150000000	0.150000000	3.66×10^{-13}
t_d^{map} , 动态表面	0.052619962	0.052619962	8.68×10^{-14}

水分单位为 kg/kg；相对差以固定域结果分母。末两行为同一固定域达标时刻的比较。

两法达标时间相对差为 5.81×10^{-13} ，全水分场最大差为 $2.61 \times 10^{-13} \text{ kg/kg}$ 、温度场最大差为 $1.16 \times 10^{-10} \text{ }^{\circ}\text{C}$ ，并通过几何守恒检查。

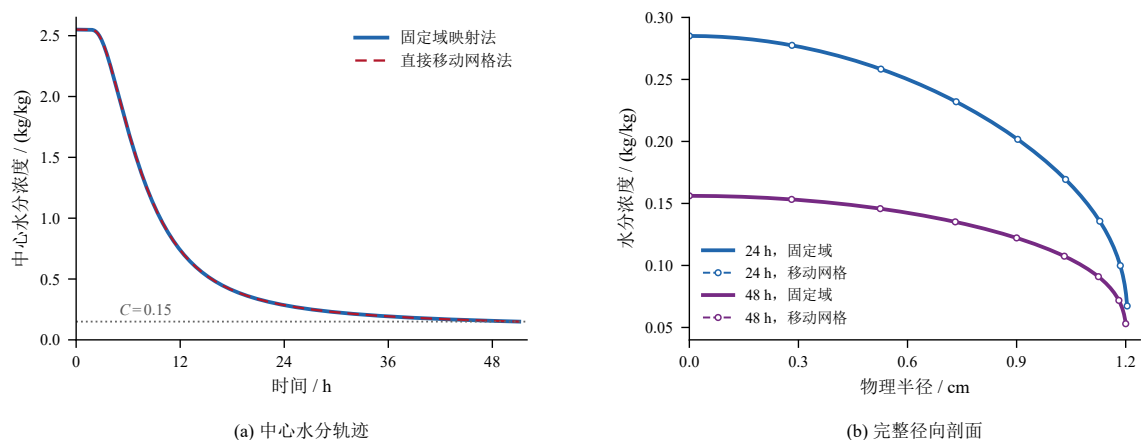


图 14 两种坐标实现的中心水分轨迹及物理半径剖面对比；曲线在图示精度下重合，圆点仅用于区分移动网格曲线

两种坐标实现的差异处于浮点舍入量级，支持程序实现的一致性；该比较不替代对收缩假设的实验验证。

9.3.6 空间网格与时间步长检验

按问题三的加密方案对收缩模型复算，结果见表 16。该检验用于评估主模型的空间和时间离散稳定性；161 至 321 节点的达标时间相对变化为 5.77×10^{-7} ，2 s 至 1 s 的变化为 1.29×10^{-5} ，满足预设容差。累计归一化水分方程残差为 1.77×10^{-13} ，有效热方程最大单步残差为 3.21×10^{-14} ；后者仅针对不含蒸发潜热的既定温度方程。

表 16 问题四网格与步长收敛检验：达标时间 (h)

计算设置	问题四
41 节点, 10 s	51.093100
81 节点, 10 s	51.093511
161 节点, 10 s	51.093626
321 节点, 10 s	51.093656
161 节点, 2 s	51.088358
161 节点, 1 s (终算)	51.087699

9.3.7 端面影响的二维检验

沿用问题一的半长圆柱轴对称模型，在侧表面与端面施加热、质 Robin 边界，并加入给定半径历史与均匀径向收缩。采用 30 s 步长， 41×21 节点下二维达标时间为 51.106259 h，与配对一维计算相对差为 1.86×10^{-7} ；加密至 81×41 节点后为 51.106671 h，相对差为 1.59×10^{-7} 。

全二维最大水分均位于轴线与中截面交点。 41×21 网格下，全过程中截面最大温差为 $0.000108\text{ }^{\circ}\text{C}$ ，最大水分差为 $1.38 \times 10^{-6}\text{ kg/kg}$ ；最大端面与中截面水分差约为 1.707 kg/kg 。该检验支持当前端面条件下的中心控制，主模型仍采用 1 s 终算时长。

9.3.8 本问结论与分析

时间步与空间网格加密的达标时间相对差均小于 1.3×10^{-5} ；结合移动网格与有限元复核，数值结果支持收缩缩短扩散距离、加快整体干燥的判断。

十、模型的评价与推广

10.1 模型的评价

模型的优点：

1. 四问采用统一的热湿迁移框架，依次引入状态相关物性、全域首达判据与材料收缩，能够衔接短期预热和长期干燥预测。
2. 有限体积法直接平衡控制体通量，全隐式与 Picard 迭代适应非线性物性；网格、步长、二维及交叉求解检验支持数值结果的可靠性。
3. 全域阈值判据识别中心控制位置，同物性固定半径对照分离几何收缩效应，便于解释干燥时间差异。

模型的局限：

1. 忽略蒸发潜热及材料各向异性，主模型只描述中部径向分布，对蒸发冷却和端面局部状态的刻画有限。
2. 长期边界采用常值延拓，收缩采用给定半径历史与均匀径向缩放；环境波动、局部开裂及长度变化可能影响预测，实际应用仍需实验标定。

10.2 模型的推广

结合少量中心温湿测量，可标定扩散与表面交换参数，利用模型估计内部达标时刻，并进一步加入能耗、品质和设备约束以优化干燥温度策略。

对短柱或端面交换显著的材料，可采用二维轴对称模型；获得潜热和吸湿数据后，可扩展为 Luikov 型热湿耦合模型 [4]。经物性与边界重新标定，该框架可用于果蔬、木材等多孔材料的干燥预测。

AI 工具使用声明

本文在思路讨论、程序实现及文字排版过程中参考了 AI 工具的辅助建议，数值结果由程序计算获得。参赛队负责模型的建立与结果的审核，并对论文内容承担最终责任，具体使用情况见支撑材料。

参考文献

- [1] Crank J. The Mathematics of Diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.
- [2] Patankar S V. Numerical Heat Transfer and Fluid Flow[M]. Washington, DC: Hemisphere, 1980.
- [3] Fritsch F N, Carlson R E. Monotone piecewise cubic interpolation[J]. SIAM Journal on Numerical Analysis, 1980, 17(2): 238–246. DOI: 10.1137/0717021.
- [4] Luikov A V. Systems of differential equations of heat and mass transfer in capillary-porous bodies[J]. International Journal of Heat and Mass Transfer, 1975, 18(1): 1–14.
- [5] Adrover A, Brasiello A. The role of shrinkage on food isothermal drying: a moving boundary model[J]. Chemical Engineering Transactions, 2019, 74: 1111–1116. DOI: 10.3303/CET1974186.

附录 A 支撑材料文件列表

表 17 支撑材料及作用

文件	作用
<code>solve_drying.py</code>	四问有限体积求解、首达判定、移动边界与步长检验主程序
<code>write_results.py</code>	四份提交工作簿的生成、格式处理和回读核验
<code>validate_2d.py</code>	二维轴对称端面效应独立核验
<code>run_sensitivity.py</code>	环境温度及水分浓度的灵敏度分析
<code>make_figures.py</code>	论文科研图的可复现绘制与矢量导出
<code>make_additional_figures.py</code>	新增三维曲面、物性云图、达标区域与收缩截面绘制
<code>result1.xlsx</code>	问题一每 1 s、0.1 cm 的温度和水分结果
<code>result2.xlsx</code>	问题二每 1 s、0.1 cm 的温度和水分结果
<code>result3.xlsx</code>	问题三每 60 s、0.1 cm 的水分结果
<code>result4.xlsx</code>	问题四每 60 s、固定位置及动态表面结果
<code>drawio/*.drawio</code>	总体技术路线与移动边界求解流程图源文件
<code>AI_USAGE.md</code>	AI 工具、使用环节、用途及人工核验记录
<code>VERIFY_REPORT.md</code>	数值、工作簿、图形和论文最终验收记录

复现时先运行 `solve_drying.py` 生成四问终算场，再运行工作簿、二维核验、灵敏度和绘图脚本。全部程序采用确定性数值计算，不含随机过程；论文附录 B 嵌入的主程序与实际运行文件一致。

附录 B 完整源程序

以下为实际运行的四问核心求解程序全文，按源文件顺序单栏连续排版；电子支撑材料中另提供原始 Python 文件。

```
"""Numerical solution for CUMCM 2026 Problem A.

The model uses a node-centred finite-volume discretisation in the radial
coordinate, fully implicit diffusion, Picard iteration for nonlinear
properties, and an ALE-style fixed-domain transform for shrinkage.
"""

from __future__ import annotations

import argparse
import json
import math
import sys
from dataclasses import dataclass
from pathlib import Path
from typing import Callable

import numpy as np
from openpyxl import load_workbook
from scipy.interpolate import PchipInterpolator
from scipy.linalg import solve_banded

ROOT = Path(__file__).resolve().parents[1]
INPUT_DIR = ROOT / "附件"
RESULTS_DIR = ROOT / "results"
H_T = 25.0
H_M = 8.0e-7
R_FIXED = 0.02
NODES = 161
OUTPUT_NODES = 21
GRID_POWER = 2.0
INITIAL_T = 28.0
INITIAL_C = 2.55
THRESHOLD_C = 0.15
TERMINAL_TEMPERATURE_C = 50.0
TERMINAL_MOISTURE = 0.05

@dataclass
class Environment:
    time_s: np.ndarray
```

```

temperature_c: np.ndarray
moisture: np.ndarray
temp_interp: PchipInterpolator
moisture_interp: PchipInterpolator

def values(
    self,
    time_s: float,
    terminal_temp_scale: float = 1.0,
    terminal_moisture_scale: float = 1.0,
) -> tuple[float, float]:
    cutoff = float(self.time_s[-1])
    if time_s <= cutoff:
        return float(self.temp_interp(time_s)), float(self.moisture_interp(time_s))
    return (
        float(TERMINAL_TEMPERATURE_C * terminal_temp_scale),
        float(TERMINAL_MOISTURE * terminal_moisture_scale),
    )

@dataclass
class RadiusHistory:
    time_s: np.ndarray
    radius_m: np.ndarray
    interp: PchipInterpolator
    derivative: PchipInterpolator
    monotone_adjustments: int

    def values(self, time_s: float) -> tuple[float, float]:
        bounded = min(max(time_s, float(self.time_s[0])), float(self.time_s[-1]))
        return float(self.interp(bounded)), float(self.derivative(bounded))

def _read_numeric_sheet(path: Path) -> np.ndarray:
    book = load_workbook(path, read_only=True, data_only=True)
    sheet = book.active
    rows: list[list[float]] = []
    for row in sheet.iter_rows(min_row=2, values_only=True):
        if row[0] is None:
            continue
        rows.append([float(value) for value in row])
    return np.asarray(rows, dtype=float)

def load_environment() -> Environment:
    data = _read_numeric_sheet(INPUT_DIR / "附件1.xlsx")
    time_s, temperature_c, moisture = data.T

```

```

if not np.all(np.diff(time_s) > 0):
    raise ValueError("附件1时间列不是严格递增。")
if not np.all(np.isfinite(data)):
    raise ValueError("附件1含非有限数。")
return Environment(
    time_s=time_s,
    temperature_c=temperature_c,
    moisture=moisture,
    temp_interp=PchipInterpolator(time_s, temperature_c),
    moisture_interp=PchipInterpolator(time_s, moisture),
)

def load_radius_history() -> RadiusHistory:
    data = _read_numeric_sheet(INPUT_DIR / "附件2.xlsx")
    time_s = data[:, 0]
    raw_radius_m = data[:, 1] / 100.0
    if not np.all(np.diff(time_s) > 0):
        raise ValueError("附件2时间列不是严格递增。")
    monotone_radius_m = np.minimum.accumulate(raw_radius_m)
    adjustments = int(np.count_nonzero(np.abs(monotone_radius_m - raw_radius_m) > 1e-12))
    interp = PchipInterpolator(time_s, monotone_radius_m, extrapolate=False)
    return RadiusHistory(
        time_s=time_s,
        radius_m=monotone_radius_m,
        interp=interp,
        derivative=interp.derivative(),
        monotone_adjustments=adjustments,
    )

def property_q1(temperature_c: np.ndarray, moisture: np.ndarray) -> tuple[np.ndarray, ...]:
    del temperature_c
    rho = np.full_like(moisture, 820.0)
    cp = np.full_like(moisture, 2600.0)
    k = np.full_like(moisture, 0.36)
    safe_c = np.maximum(moisture, 1e-8)
    diffusivity = 7.0e-9 * np.exp(-0.89 / safe_c)
    return rho, cp, k, diffusivity

def property_q23(temperature_c: np.ndarray, moisture: np.ndarray) -> tuple[np.ndarray, ...]:
    safe_c = np.maximum(moisture, 1e-8)
    rho = 650.0 + 128.0 * safe_c
    cp = 1450.0 + 2736.0 * safe_c / (safe_c + 1.0)
    k = 0.21 + 0.38 * safe_c / (safe_c + 1.0)
    diffusivity = (

```

```

        2.4e-3
        * np.exp(-0.45 / safe_c)
        * np.exp(-3850.0 / (temperature_c + 273.15))
    )
    return rho, cp, k, diffusivity

def property_q4(temperature_c: np.ndarray, moisture: np.ndarray) -> tuple[np.ndarray, ...]:
    safe_c = np.maximum(moisture, 1e-8)
    rho = 760.0 + 90.0 * safe_c
    cp = 1850.0 + 2150.0 * safe_c / (safe_c + 1.0)
    k = 0.12 + 0.20 * safe_c / (safe_c + 1.0)
    diffusivity = (
        4.2e-4
        * np.exp(-0.30 / safe_c)
        * np.exp(-3850.0 / (temperature_c + 273.15))
    )
    return rho, cp, k, diffusivity

def face_interpolate(left: np.ndarray, right: np.ndarray) -> np.ndarray:
    """Linear face interpolation for a smooth state-dependent coefficient."""
    return 0.5 * (left + right)

def radial_grid(count: int, radius: float = 1.0) -> np.ndarray:
    """Surface-refined computational mesh; output positions are independent."""
    s = np.linspace(0.0, 1.0, count)
    return radius * (1.0 - (1.0 - s) ** GRID_POWER)

def node_control_volumes(coordinate: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Return dimensionless/physical radial CV volumes and internal face radii."""
    faces = 0.5 * (coordinate[:-1] + coordinate[1:])
    lower = np.concatenate(([0.0], faces))
    upper = np.concatenate((faces, [coordinate[-1]]))
    volumes = math.pi * (upper**2 - lower**2)
    return volumes, faces

def implicit_radial_step(
    old: np.ndarray,
    storage: np.ndarray,
    conductivity: np.ndarray,
    dt: float,
    coordinate: np.ndarray,
    diffusion_scale: float,

```

```

    boundary_conductance: float,
    boundary_value: float,
    advection_velocity: np.ndarray | None = None,
) -> np.ndarray:
    """One implicit node-centred radial FVM step on coordinate [0, 1] or [0, R]."""
    count = len(old)
    volumes, faces = node_control_volumes(coordinate)
    delta = np.diff(coordinate)
    face_property = face_interpolate(conductivity[:-1], conductivity[1:])
    conductance = 2.0 * math.pi * faces * face_property * diffusion_scale / delta

    capacity = storage * volumes / dt
    diagonal = capacity.copy()
    lower = np.zeros(count - 1)
    upper = np.zeros(count - 1)
    rhs = capacity * old

    diagonal[:-1] += conductance
    diagonal[1:] += conductance
    upper -= conductance
    lower -= conductance

    diagonal[-1] += boundary_conductance
    rhs[-1] += boundary_conductance * boundary_value

    if advection_velocity is not None:
        # PDE form:  $\phi_t + u(x_i) \phi_{x_i} = \text{diffusion}$ ;  $u \geq 0$  under shrinkage.
        advective = storage[1:] * volumes[1:] * np.maximum(advection_velocity[1:], 0.0) / delta
        diagonal[1:] += advective
        lower -= advective

    matrix = np.zeros((3, count))
    matrix[0, 1:] = upper
    matrix[1, :] = diagonal
    matrix[2, :-1] = lower
    return solve_banded((1, 1), matrix, rhs, check_finite=False)

def advance_coupled(
    temperature_old: np.ndarray,
    moisture_old: np.ndarray,
    dt: float,
    radius_m: float,
    radius_rate_m_s: float,
    ambient_temperature_c: float,
    ambient_moisture: float,
    property_model: Callable[[np.ndarray, np.ndarray], tuple[np.ndarray, ...]],

```

```

moving_domain: bool,
tolerance_temperature: float = 1e-8,
tolerance_moisture: float = 1e-10,
max_iterations: int = 30,
balance: dict | None = None,
) -> tuple[np.ndarray, np.ndarray, int]:
    temperature = temperature_old.copy()
    moisture = moisture_old.copy()
    if moving_domain:
        coordinate = radial_grid(len(temperature_old))
        diffusion_scale = 1.0 / radius_m**2
        boundary_scale = 1.0 / radius_m
        shrink_velocity = -(radius_rate_m_s / radius_m) * coordinate
    else:
        coordinate = radial_grid(len(temperature_old), radius_m)
        diffusion_scale = 1.0
        boundary_scale = 1.0
        shrink_velocity = None

    for iteration in range(1, max_iterations + 1):
        rho, cp, k, _ = property_model(temperature, moisture)
        heat_storage = rho * cp
        heat_boundary = 2.0 * math.pi * coordinate[-1] * H_T * boundary_scale
        next_temperature = implicit_radial_step(
            old=temperature_old,
            storage=heat_storage,
            conductivity=k,
            dt=dt,
            coordinate=coordinate,
            diffusion_scale=diffusion_scale,
            boundary_conductance=heat_boundary,
            boundary_value=ambient_temperature_c,
            advection_velocity=shrink_velocity,
        )

        _, _, _, diffusivity = property_model(next_temperature, moisture)
        mass_boundary = 2.0 * math.pi * coordinate[-1] * H_M * boundary_scale
        next_moisture = implicit_radial_step(
            old=moisture_old,
            storage=np.ones_like(moisture),
            conductivity=diffusivity,
            dt=dt,
            coordinate=coordinate,
            diffusion_scale=diffusion_scale,
            boundary_conductance=mass_boundary,
            boundary_value=ambient_moisture,
            advection_velocity=shrink_velocity,

```

```

    )

    error_t = float(np.max(np.abs(next_temperature - temperature)))
    error_c = float(np.max(np.abs(next_moisture - moisture)))
    temperature = next_temperature
    moisture = next_moisture
    if error_t < tolerance_temperature and error_c < tolerance_moisture:
        break
else:
    raise RuntimeError(
        f"Picard iteration failed: dT={error_t:.3e}, dC={error_c:.3e}, dt={dt}"
    )

if not np.all(np.isfinite(temperature)) or not np.all(np.isfinite(moisture)):
    raise FloatingPointError("Non-finite state generated.")
if np.min(moisture) < -1e-10 or np.max(moisture) > INITIAL_C + 1e-8:
    raise FloatingPointError("Moisture left physical bounds.")
if balance is not None:
    volumes, _ = node_control_volumes(coordinate)
    adv_c = adv_t = 0.0
    if shrink_velocity is not None:
        weight = volumes[1:] * np.maximum(shrink_velocity[1:], 0.0) / np.diff(coordinate)
        adv_c = dt * float(np.dot(weight, np.diff(moisture)))
        adv_t = dt * float(np.dot(weight * heat_storage[1:], np.diff(temperature)))
    change = float(np.dot(volumes, moisture - moisture_old))
    flux = dt * mass_boundary * (float(moisture[-1]) - ambient_moisture)
    reference = INITIAL_C * float(volumes.sum())
    balance['moisture_change'] += change / reference
    balance['boundary_loss'] += flux / reference
    balance['relative_transport'] += adv_c / reference
    residual = change + flux + adv_c
    balance['maximum_step_mass_residual'] = max(balance['maximum_step_mass_residual'],
abs(residual) / reference)
    heat_change = float(np.dot(heat_storage * volumes, temperature - temperature_old))
    heat_flux = dt * heat_boundary * (float(temperature[-1]) - ambient_temperature_c)
    heat_reference = INITIAL_T * float(np.dot(heat_storage, volumes))
    heat_residual = (heat_change + heat_flux + adv_t) / heat_reference
    balance['maximum_step_heat_equation_residual'] =
max(balance['maximum_step_heat_equation_residual'], abs(heat_residual))
return temperature, moisture, iteration

def new_balance() -> dict:
    return dict(moisture_change=0.0, boundary_loss=0.0, relative_transport=0.0,
                maximum_step_mass_residual=0.0, maximum_step_heat_equation_residual=0.0)

```

```

def simulate_fixed(
    environment: Environment,
    property_model: Callable[[np.ndarray, np.ndarray], tuple[np.ndarray, ...]],
    end_time_s: float | None,
    dt_s: float,
    output_interval_s: float,
    stop_at_threshold: bool,
    terminal_temp_scale: float = 1.0,
    terminal_moisture_scale: float = 1.0,
    stop_interval_s: float | None = None,
) -> dict[str, np.ndarray | float | int | bool]:
    temperature = np.full(NODES, INITIAL_T)
    moisture = np.full(NODES, INITIAL_C)
    radial_m = radial_grid(NODES, R_FIXED)
    output_r = np.linspace(0.0, R_FIXED, OUTPUT_NODES)
    times = [0.0]
    temperatures = [np.interp(output_r, radial_m, temperature)]
    moistures = [np.interp(output_r, radial_m, moisture)]
    iterations: list[int] = []
    monotone_profiles = True
    center_is_max = True
    previous_max = float(np.max(moisture))
    previous_time = 0.0
    threshold_time = math.nan
    threshold_temperature: np.ndarray | None = None
    threshold_moisture: np.ndarray | None = None
    stop_output_time = math.inf
    balance = new_balance()
    time_s = 0.0
    next_output = output_interval_s
    maximum_time = float(end_time_s if end_time_s is not None else 10.0 * 24.0 * 3600.0)

    while time_s < maximum_time - 1e-9:
        step = min(dt_s, maximum_time - time_s, next_output - time_s)
        new_time = time_s + step
        temperature_before = temperature.copy()
        moisture_before = moisture.copy()
        ambient_t, ambient_c = environment.values(
            new_time, terminal_temp_scale, terminal_moisture_scale
        )
        temperature, moisture, count = advance_coupled(
            temperature,
            moisture,
            step,
            R_FIXED,
            0.0,
            ambient_t,

```

```

        ambient_c,
        property_model,
        moving_domain=False,
        balance=balance,
    )
    iterations.append(count)
    time_s = new_time
    current_max = float(np.max(moisture))
    monotone_profiles = monotone_profiles and bool(np.all(np.diff(moisture) <= 2e-9))
    center_is_max = center_is_max and bool(
        moisture[0] >= float(np.max(moisture)) - 1e-9
    )

    if previous_max > THRESHOLD_C >= current_max and math.isnan(threshold_time):
        fraction = (previous_max - THRESHOLD_C) / max(previous_max - current_max, 1e-15)
        threshold_time = previous_time + fraction * step
        threshold_temperature = (
            temperature_before + fraction * (temperature - temperature_before)
        )
        threshold_moisture = moisture_before + fraction * (moisture - moisture_before)
        stop_spacing = output_interval_s if stop_interval_s is None else stop_interval_s
        stop_output_time = math.ceil(threshold_time / stop_spacing) * stop_spacing

    if time_s + 1e-8 >= next_output:
        times.append(time_s)
        temperatures.append(np.interp(output_r, radial_m, temperature))
        moistures.append(np.interp(output_r, radial_m, moisture))
        next_output += output_interval_s

    if stop_at_threshold and time_s + 1e-8 >= stop_output_time:
        break

    previous_max = current_max
    previous_time = time_s

    return {
        "times_s": np.asarray(times),
        "radius_m": output_r,
        "temperature_c": np.asarray(temperatures),
        "moisture": np.asarray(moistures),
        "threshold_time_s": threshold_time,
        "threshold_temperature_c": None if threshold_temperature is None else
        np.interp(output_r, radial_m, threshold_temperature),
        "threshold_moisture": None if threshold_moisture is None else np.interp(output_r,
        radial_m, threshold_moisture),
        "last_time_s": time_s,
        "picard_max": int(max(iterations, default=0)),
        "picard_mean": float(np.mean(iterations) if iterations else 0.0),
    }

```

```

        "monotone_profiles": monotone_profiles,
        "center_is_max": center_is_max,
        "balance": balance,
    }

def simulate_moving(
    environment: Environment,
    radius_history: RadiusHistory,
    dt_s: float,
    output_interval_s: float,
    stop_at_threshold: bool = True,
    terminal_temp_scale: float = 1.0,
    terminal_moisture_scale: float = 1.0,
    include_advection: bool = False,
) -> dict[str, np.ndarray | float | int | bool]:
    # Default: homogeneous solid shrinkage, material velocity equals mesh velocity.
    # True retains the former stationary-spatial scalar model for structural comparison.
    xi = radial_grid(NODES)
    # Preserve native nodes; interpolate to physical positions only at export.
    output_xi = xi.copy()
    temperature = np.full(NODES, INITIAL_T)
    moisture = np.full(NODES, INITIAL_C)
    times = [0.0]
    radii = [R_FIXED]
    temperatures = [np.interp(output_xi, xi, temperature)]
    moistures = [np.interp(output_xi, xi, moisture)]
    iterations: list[int] = []
    previous_max = float(np.max(moisture))
    previous_time = 0.0
    threshold_time = math.nan
    threshold_temperature: np.ndarray | None = None
    threshold_moisture: np.ndarray | None = None
    threshold_radius_m = math.nan
    stop_output_time = math.inf
    time_s = 0.0
    next_output = output_interval_s
    maximum_time = float(radius_history.time_s[-1])
    monotone_profiles = True
    center_is_max = True
    maximum_cfl = 0.0
    balance = new_balance()

    while time_s < maximum_time - 1e-9:
        step = min(dt_s, maximum_time - time_s, next_output - time_s)
        new_time = time_s + step
        temperature_before = temperature.copy()

```

```

moisture_before = moisture.copy()
radius_m, radius_rate_m_s = radius_history.values(new_time)
if not include_advection:
    radius_rate_m_s = 0.0
ambient_t, ambient_c = environment.values(
    new_time, terminal_temp_scale, terminal_moisture_scale
)
temperature, moisture, count = advance_coupled(
    temperature,
    moisture,
    step,
    radius_m,
    radius_rate_m_s,
    ambient_t,
    ambient_c,
    property_q4,
    moving_domain=True,
    balance=balance,
)
iterations.append(count)
time_s = new_time
current_max = float(np.max(moisture))
maximum_cfl = max(
    maximum_cfl,
    float(np.max(abs(radius_rate_m_s / radius_m) * xi[1:] * step / np.diff(xi))),
)
monotone_profiles = monotone_profiles and bool(np.all(np.diff(moisture) <= 2e-9))
center_is_max = center_is_max and bool(
    moisture[0] >= float(np.max(moisture)) - 1e-9
)

if previous_max > THRESHOLD_C >= current_max and math.isnan(threshold_time):
    fraction = (previous_max - THRESHOLD_C) / max(previous_max - current_max, 1e-15)
    threshold_time = previous_time + fraction * step
    threshold_temperature = (
        temperature_before + fraction * (temperature - temperature_before)
    )
    threshold_moisture = moisture_before + fraction * (moisture - moisture_before)
    threshold_radius_m = radius_history.values(threshold_time)[0]
    stop_output_time = (
        math.ceil(threshold_time / output_interval_s) * output_interval_s
    )

if time_s + 1e-8 >= next_output:
    times.append(time_s)
    radii.append(radius_m)
    temperatures.append(np.interp(output_xi, xi, temperature))

```

```

        moistures.append(np.interp(output_xi, xi, moisture))
        next_output += output_interval_s

    if stop_at_threshold and time_s + 1e-8 >= stop_output_time:
        break

    previous_max = current_max
    previous_time = time_s

return {
    "times_s": np.asarray(times),
    "xi": output_xi,
    "radii_m": np.asarray(radii),
    "temperature_c": np.asarray(temperatures),
    "moisture": np.asarray(moistures),
    "threshold_time_s": threshold_time,
    "threshold_temperature_c": None if threshold_temperature is None else
np.interp(output_xi, xi, threshold_temperature),
    "threshold_moisture": None if threshold_moisture is None else np.interp(output_xi, xi,
threshold_moisture),
    "threshold_radius_m": threshold_radius_m,
    "last_time_s": time_s,
    "picard_max": int(max(iterations, default=0)),
    "picard_mean": float(np.mean(iterations) if iterations else 0.0),
    "monotone_profiles": monotone_profiles,
    "center_is_max": center_is_max,
    "maximum_advection_cfl": maximum_cfl,
    "balance": balance,
    "velocity_model": "stationary_spatial_comparison" if include_advection else
    "uniform_material_shrinkage",
}

def extract_fixed_table(
    simulation: dict[str, np.ndarray | float | int | bool],
    requested_times_s: list[float],
    requested_radii_cm: list[float],
    variable: str,
) -> list[list[float]]:
    times = np.asarray(simulation["times_s"])
    radii_m = np.asarray(simulation["radius_m"])
    values = np.asarray(simulation[variable])
    table: list[list[float]] = []
    for requested_time in requested_times_s:
        threshold_time = float(simulation.get("threshold_time_s", math.nan))
        threshold_key = (
            "threshold_temperature_c" if variable == "temperature_c" else "threshold_moisture"
        )

```

```

threshold_profile = simulation.get(threshold_key)
if (
    threshold_profile is not None
    and math.isfinite(threshold_time)
    and abs(requested_time - threshold_time) < 1e-5
):
    profile = np.asarray(threshold_profile)
else:
    time_index = int(np.argmin(np.abs(times - requested_time)))
    profile = values[time_index]
row = [
    float(np.interp(radius_cm / 100.0, radii_m, profile))
    for radius_cm in requested_radii_cm
]
table.append(row)
return table

def extract_moving_profile(
    simulation: dict[str, np.ndarray | float | int | bool],
    requested_time_s: float,
    requested_radii_cm: list[float],
) -> list[float | None]:
    times = np.asarray(simulation["times_s"])
    xi = np.asarray(simulation["xi"])
    radii_m = np.asarray(simulation["radii_m"])
    moisture = np.asarray(simulation["moisture"])
    threshold_time = float(simulation.get("threshold_time_s", math.nan))
    threshold_profile = simulation.get("threshold_moisture")
    if (
        threshold_profile is not None
        and math.isfinite(threshold_time)
        and abs(requested_time_s - threshold_time) < 1e-5
    ):
        profile = np.asarray(threshold_profile)
        radius = float(simulation["threshold_radius_m"])
    else:
        index = int(np.argmin(np.abs(times - requested_time_s)))
        profile = moisture[index]
        radius = float(radii_m[index])
    values: list[float | None] = []
    for radius_cm in requested_radii_cm:
        position_m = radius_cm / 100.0
        if position_m > radius + 1e-12:
            values.append(None)
        else:
            values.append(float(np.interp(position_m / radius, xi, profile)))

```

```

        values.append(float(profile[-1]))
    return values

def save_npz(path: Path, simulation: dict[str, np.ndarray | float | int | bool]) -> None:
    arrays = {key: value for key, value in simulation.items() if isinstance(value, np.ndarray)}
    np.savez_compressed(path, **arrays)

def json_ready(value):
    if isinstance(value, np.ndarray):
        return value.tolist()
    if isinstance(value, (np.floating, np.integer)):
        return value.item()
    if isinstance(value, float) and math.isnan(value):
        return None
    if isinstance(value, dict):
        return {key: json_ready(item) for key, item in value.items()}
    if isinstance(value, (list, tuple)):
        return [json_ready(item) for item in value]
    return value

def write_summary(path: Path, payload: dict) -> None:
    path.write_text(
        json.dumps(json_ready(payload), ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

def run_all(run_convergence: bool = True) -> None:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    for name in ("q1", "q2", "q3", "q4"):
        (RESULTS_DIR / name).mkdir(parents=True, exist_ok=True)

    environment = load_environment()
    radius_history = load_radius_history()
    data_audit = {
        "environment_rows": len(environment.time_s),
        "environment_time_range_s": [environment.time_s[0], environment.time_s[-1]],
        "environment_sampling_s": np.unique(np.diff(environment.time_s)),
        "temperature_range_c": [
            float(np.min(environment.temperature_c)),
            float(np.max(environment.temperature_c)),
        ],
        "ambient_moisture_range": [
            float(np.min(environment.moisture)),

```

```

        float(np.max(environment.moisture)),
    ],
    "terminal_boundary_after_14400_s": {
        "temperature_c": TERMINAL_TEMPERATURE_C,
        "moisture": TERMINAL_MOISTURE,
        "basis": "constant-stage setpoints; final-hour attachment means are 49.9989 C and
0.049988",
    },
    "radius_rows": len(radius_history.time_s),
    "radius_time_range_s": [radius_history.time_s[0], radius_history.time_s[-1]],
    "radius_range_cm": [
        float(np.min(radius_history.radius_m) * 100.0),
        float(np.max(radius_history.radius_m) * 100.0),
    ],
    "radius_monotone_adjustments": radius_history.monotone_adjustments,
}
write_summary(RESULTS_DIR / "data_audit.json", data_audit)

q1 = simulate_fixed(
    environment,
    property_q1,
    end_time_s=1800.0,
    dt_s=1.0,
    output_interval_s=1.0,
    stop_at_threshold=False,
)
save_npz(RESULTS_DIR / "q1" / "fields.npz", q1)
q1_times = [100, 300, 600, 900, 1200, 1500, 1800]
table_radai = [0.0, 0.5, 1.0, 1.5, 2.0]
q1_summary = {
    "table_times_s": q1_times,
    "table_radai_cm": table_radai,
    "temperature_table_c": extract_fixed_table(q1, q1_times, table_radai, "temperature_c"),
    "moisture_table": extract_fixed_table(q1, q1_times, table_radai, "moisture"),
    "biot_heat": H_T * R_FIXED / 0.36,
    "initial_diffusivity_m2_s": float(property_q1(
        np.array([INITIAL_T]), np.array([INITIAL_C])
    )[3][0]),
    "biot_mass_initial": H_M * R_FIXED / float(property_q1(
        np.array([INITIAL_T]), np.array([INITIAL_C])
    )[3][0]),
    "picard_max": q1["picard_max"],
    "picard_mean": q1["picard_mean"],
    "temperature_bounds_c": [
        float(np.min(q1["temperature_c"])),
        float(np.max(q1["temperature_c"])),
    ],
}

```

```

        "moisture_bounds": [
            float(np.min(q1["moisture"])),
            float(np.max(q1["moisture"])),
        ],
    }
    write_summary(RESULTS_DIR / "q1" / "summary.json", q1_summary)

    q2 = simulate_fixed(
        environment,
        property_q23,
        end_time_s=None,
        dt_s=1.0,
        output_interval_s=1.0,
        stop_at_threshold=True,
        stop_interval_s=60.0,
    )
    save_npz(RESULTS_DIR / "q2" / "fields.npz", q2)
    q2_times = [1800, 3600, 5400, 7200, 9000, 10800]
    q2_summary = {
        "table_times_h": [time / 3600.0 for time in q2_times],
        "table_radai_cm": table_radai,
        "temperature_table_c": extract_fixed_table(q2, q2_times, table_radai, "temperature_c"),
        "moisture_table": extract_fixed_table(q2, q2_times, table_radai, "moisture"),
        "picard_max": q2["picard_max"],
        "picard_mean": q2["picard_mean"],
        "monotone_profiles": q2["monotone_profiles"],
        "center_is_max": q2["center_is_max"],
    }
    write_summary(RESULTS_DIR / "q2" / "summary.json", q2_summary)

    q3 = dict(q2)
    for key in ("times_s", "temperature_c", "moisture"):
        q3[key] = q2[key][:60].copy()
    save_npz(RESULTS_DIR / "q3" / "fields.npz", q3)
    q3_end = float(q3["threshold_time_s"])
    q3_report_times = list(np.arange(6 * 3600, math.floor(q3_end / (6 * 3600)) * 6 * 3600 + 1,
        6 * 3600))
    if not q3_report_times or abs(q3_report_times[-1] - q3_end) > 1e-6:
        q3_report_times.append(q3_end)
    q3_summary = {
        "drying_time_s": q3_end,
        "drying_time_h": q3_end / 3600.0,
        "output_end_s": q3["last_time_s"],
        "table_times_h": [time / 3600.0 for time in q3_report_times],
        "table_radai_cm": table_radai,
        "moisture_table": extract_fixed_table(q3, q3_report_times, table_radai, "moisture"),
        "center_is_max": q3["center_is_max"],
    }

```

```

        "monotone_profiles": q3["monotone_profiles"],
        "picard_max": q3["picard_max"],
        "picard_mean": q3["picard_mean"],
    }

    q4 = simulate_moving(
        environment,
        radius_history,
        dt_s=1.0,
        output_interval_s=60.0,
        stop_at_threshold=True,
    )

    save_npz(RESULTS_DIR / "q4" / "fields.npz", q4)
    q4_end = float(q4["threshold_time_s"])
    q4_report_times = list(np.arange(6 * 3600, math.floor(q4_end / (6 * 3600)) * 6 * 3600 + 1,
        6 * 3600))
    if not q4_report_times or abs(q4_report_times[-1] - q4_end) > 1e-6:
        q4_report_times.append(q4_end)
    q4_table_radii = [0.0, 0.5, 1.0]
    q4_summary = {
        "drying_time_s": q4_end,
        "drying_time_h": q4_end / 3600.0,
        "output_end_s": q4["last_time_s"],
        "table_times_h": [time / 3600.0 for time in q4_report_times],
        "table_radii_cm": q4_table_radii,
        "columns": q4_table_radii + ["药材表面"],
        "moisture_table": [
            extract_moving_profile(q4, time, q4_table_radii) for time in q4_report_times
        ],
        "radius_at_end_cm": float(radius_history.values(q4_end)[0] * 100.0),
        "center_is_max": q4["center_is_max"],
        "monotone_profiles": q4["monotone_profiles"],
        "maximum_advection_cfl": q4["maximum_advection_cfl"],
        "picard_max": q4["picard_max"],
        "picard_mean": q4["picard_mean"],
    }

    if run_convergence:
        q3_dt2 = simulate_fixed(
            environment,
            property_q23,
            end_time_s=None,
            dt_s=2.0,
            output_interval_s=60.0,
            stop_at_threshold=True,
        )
        q4_dt2 = simulate_moving(

```

```

        environment,
        radius_history,
        dt_s=2.0,
        output_interval_s=60.0,
        stop_at_threshold=True,
    )
    q4_stationary = simulate_moving(
        environment,
        radius_history,
        dt_s=1.0,
        output_interval_s=60.0,
        stop_at_threshold=True,
        include_advection=True,
    )
    q3_summary["time_step_check"] = {
        "dt1_s_time_h": q3_end / 3600.0,
        "dt2_s_time_h": float(q3_dt2["threshold_time_s"]) / 3600.0,
        "relative_difference": abs(
            float(q3_dt2["threshold_time_s"]) - q3_end
        ) / float(q3_dt2["threshold_time_s"]),
    }
    q4_summary["time_step_check"] = {
        "dt1_s_time_h": q4_end / 3600.0,
        "dt2_s_time_h": float(q4_dt2["threshold_time_s"]) / 3600.0,
        "relative_difference": abs(
            float(q4_dt2["threshold_time_s"]) - q4_end
        ) / float(q4_dt2["threshold_time_s"]),
    }
    q4_summary["moving_term_comparison"] = {
        "material_model_h": q4_end / 3600.0,
        "stationary_spatial_model_h": float(q4_stationary["threshold_time_s"]) / 3600.0,
        "difference_h": (
            float(q4_stationary["threshold_time_s"]) - q4_end
        ) / 3600.0,
    }

for summary, simulation in [(q1_summary, q1), (q2_summary, q2), (q3_summary, q3),
(q4_summary, q4)]:
    summary['numerics'] = {'nodes': NODES, 'grid_power': GRID_POWER, 'dt_s': 1.0,
'output_nodes': simulation['moisture'].shape[1]}
    summary['balance'] = simulation['balance']
    summary['balance']['cumulative_mass_equation_residual'] =
abs(sum(simulation['balance'][key] for key in
['moisture_change', 'boundary_loss', 'relative_transport']))
    q2_summary['output_end_s'] = q2['last_time_s']
    q2_summary['drying_time_h'] = q2['threshold_time_s'] / 3600.0
    q3_summary['shared_trajectory_with_q2'] = bool(np.array_equal(q3['moisture'],

```

```

q2['moisture'][:60]))
q4_summary['velocity_model'] = q4['velocity_model']
write_summary(RESULTS_DIR / "q1" / "summary.json", q1_summary)
write_summary(RESULTS_DIR / "q2" / "summary.json", q2_summary)
write_summary(RESULTS_DIR / "q3" / "summary.json", q3_summary)
write_summary(RESULTS_DIR / "q4" / "summary.json", q4_summary)

print(json.dumps(
    {
        "q1_surface_temperature_1800_c": q1_summary["temperature_table_c"][-1][-1],
        "q1_center_moisture_1800": q1_summary["moisture_table"][-1][0],
        "q2_center_temperature_3h_c": q2_summary["temperature_table_c"][-1][0],
        "q2_center_moisture_3h": q2_summary["moisture_table"][-1][0],
        "q3_drying_time_h": q3_summary["drying_time_h"],
        "q4_drying_time_h": q4_summary["drying_time_h"],
        "q4_radius_at_end_cm": q4_summary["radius_at_end_cm"],
    },
    ensure_ascii=False,
    indent=2,
))

def main() -> None:
    sys.stdout.reconfigure(encoding="utf-8")
    parser = argparse.ArgumentParser()
    parser.add_argument("--skip-convergence", action="store_true")
    args = parser.parse_args()
    run_all(run_convergence=not args.skip_convergence)

if __name__ == "__main__":
    main()

```